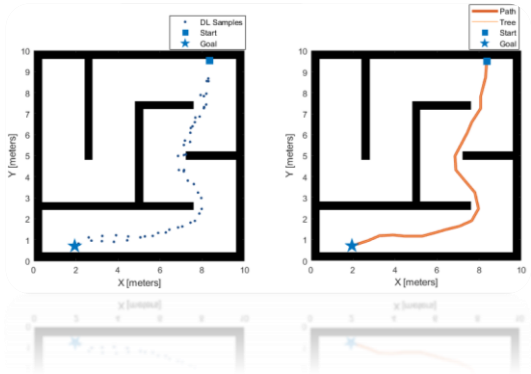
AERO60492 – Autonomous Mobile Robots

Lecture 9+10 – Path Planning

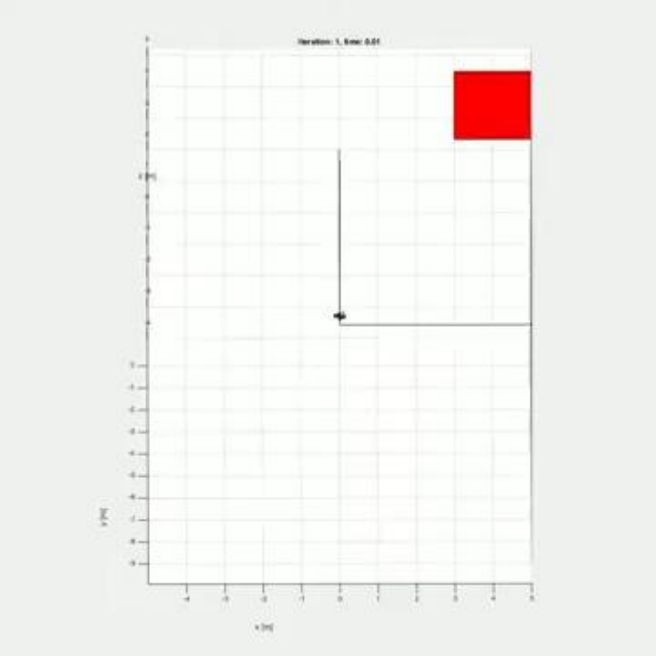
kieran.wood@manchester.ac.uk



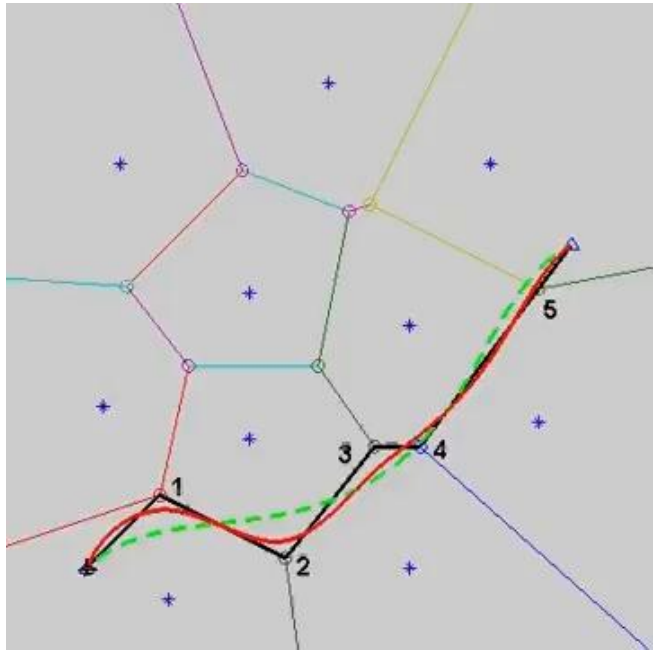
Aims

- Introduction to path planning algorithms
- Terminology
- Two popular examples
  - A*
  - RRT
- Alternative names
  - Path finding
  - Motion planning
  - Routing

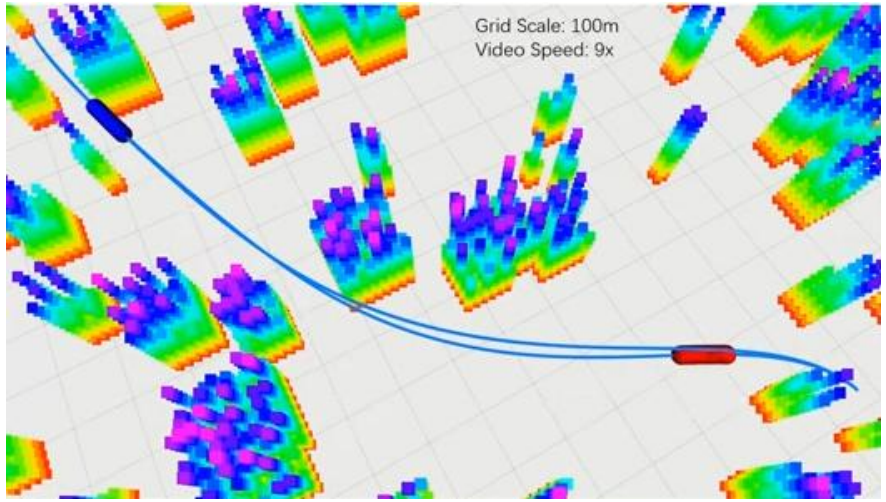
Many vairaions



A* Path Planning + Minimum Snap Trajectory Generation

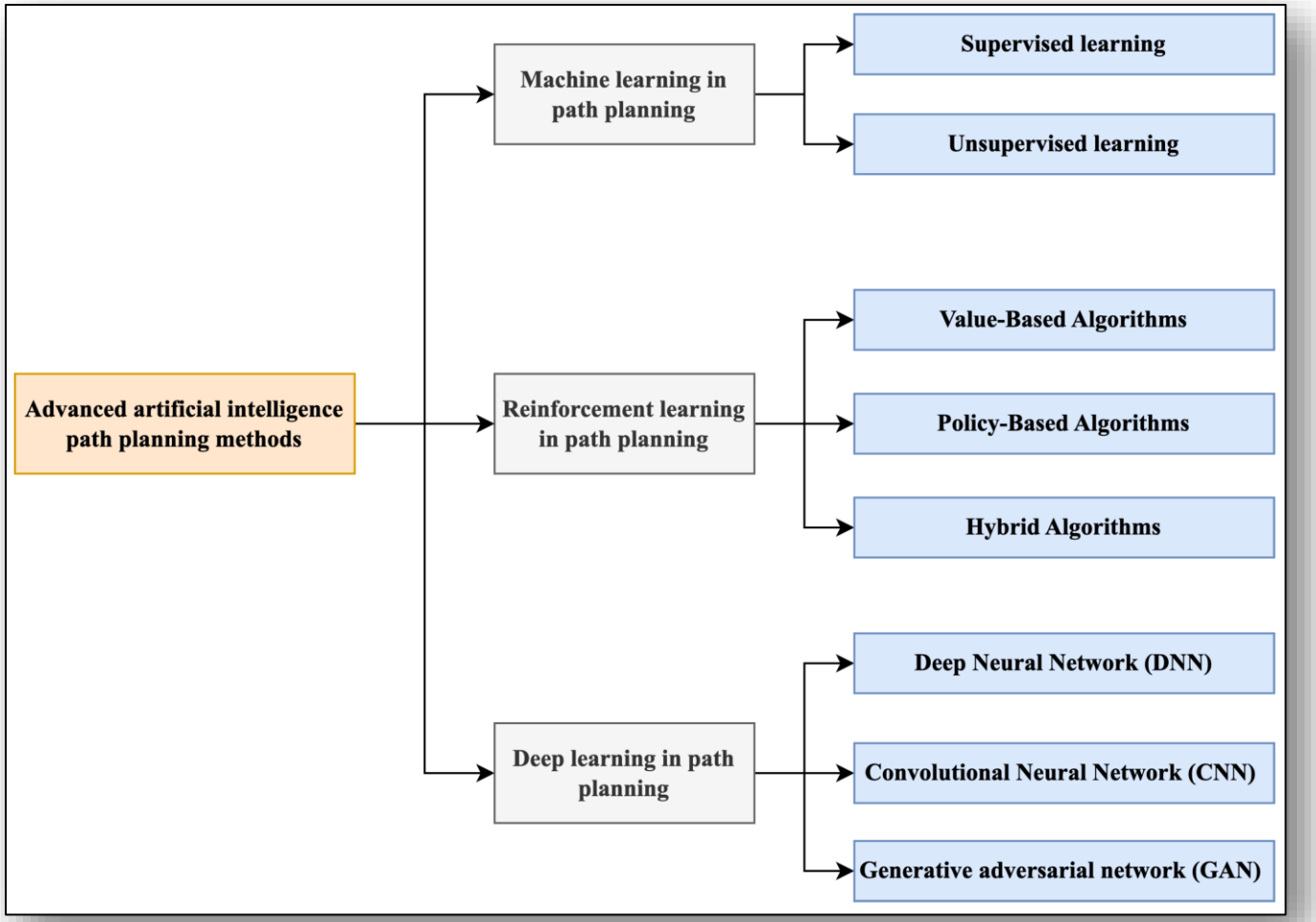


UAV path planning avoiding hazards



Robust Trajectory Planning for Spatial-Temporal Multi-Drone Coordination in Large Scenes

Not Covering AI-based methods

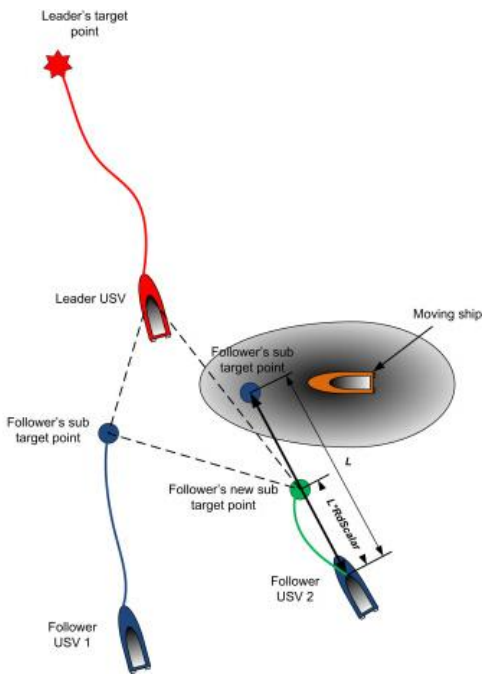


W. Meng, X. Zhang, L. Zhou, H. Guo, and X. Hu, “Advances in UAV path planning: A comprehensive review of methods, challenges, and future directions,” Drones, vol. 9, no. 5, p. 376, May 2025, doi: 10.3390/drones9050376.

Why path planning

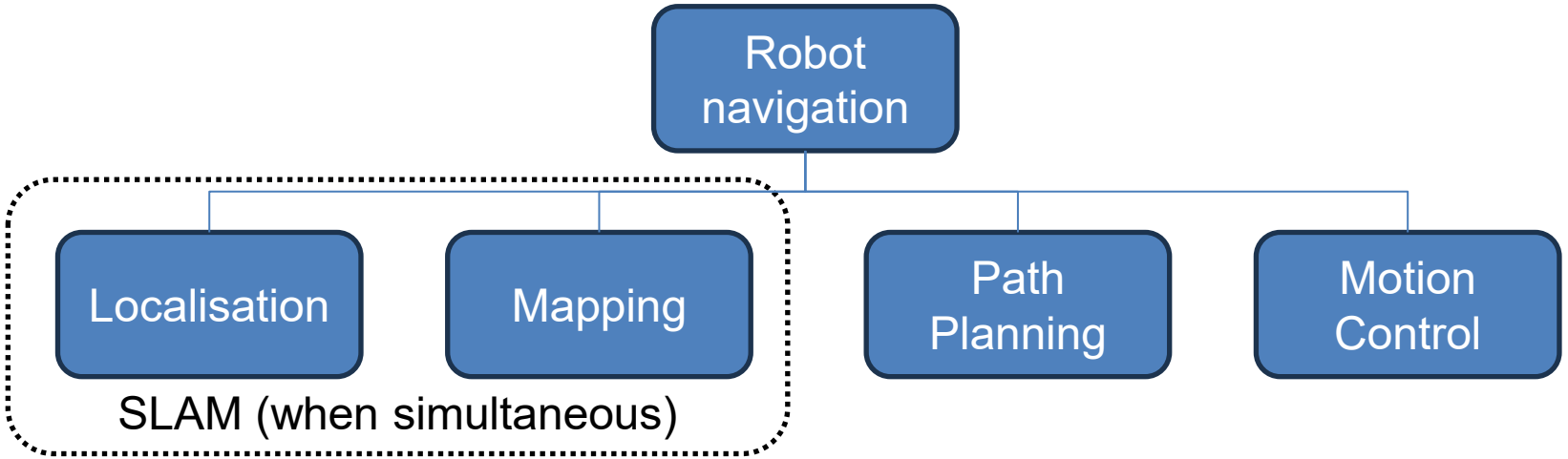
- One of the defining features of autonomous mobility is the ability to freely move within a motion envelope
  - 2D for surface (rovers, walkers, boats)
  - 3D for aerial or sub aquatic (UAV, UUV/ROV)
- Typically want to move between points whilst avoiding collisions along the way.
  - Can be optimised for efficiency, reliability, simplicity via additional cost functions (heuristics), time dependencies, formations, …

An example of formation surface path panning for Autonomous Surface Vessels taking into account obstacles of other vessels.



https://doi.org/10.1016/j.oceaneng.2015.01.008

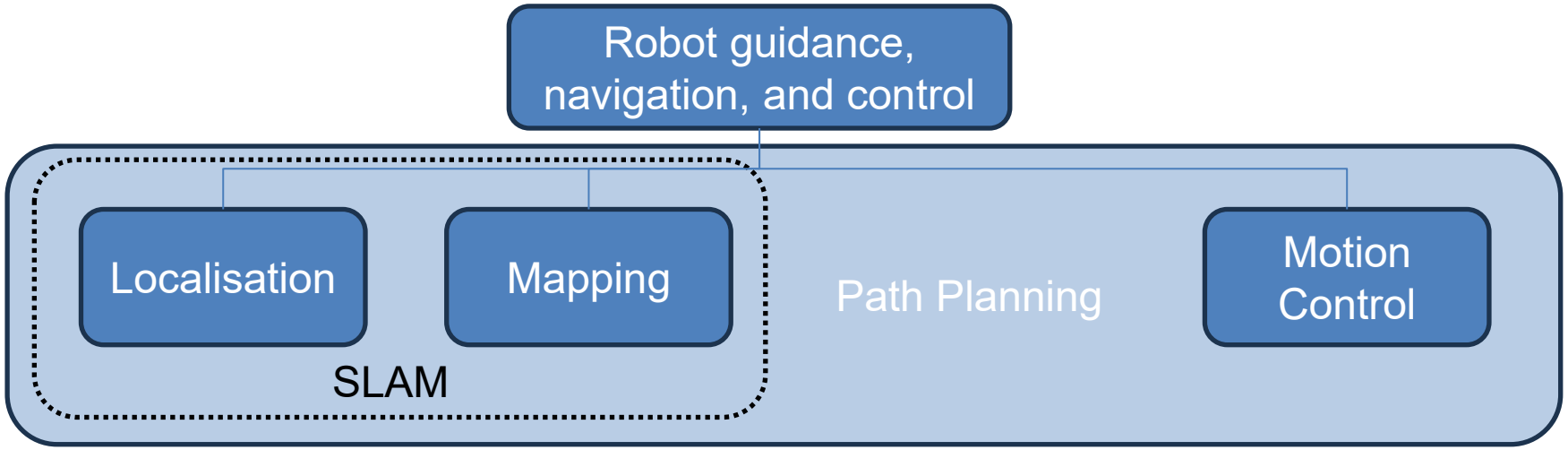
Challenges



- Motion control – moving the robot along a defined trajectory
- Mapping – a virtual representation of the local environment
- Localisation – knowing where the robot is within the map/frame
- Path planning – creating a planned route of motion

*SLAM – simultaneous localisation and mapping

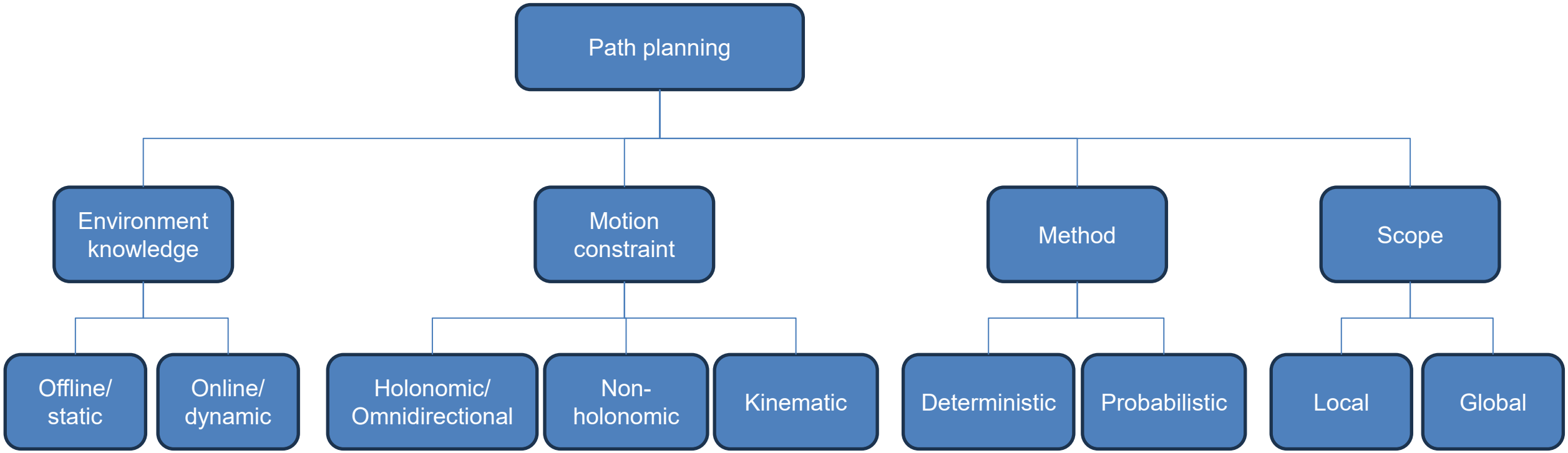
Challenges



Path planning algorithms require knowledge of:

- Where to start and stop (localisation)
- The obstacles and hazards in the area (mapping)
- What motions are allowed (motion control)

Categories



All planned before motion begins. Requires complete knowledge.

Planned incrementally and can react to changing environment or new measurements.

Vehicle can move in any direction (within its dimensionality 2D/3D)

Vehicle has constraints on its directions of movements

Vehicle has constraints on the timings or dynamics of movements

Given identical conditions, generated path will be identical

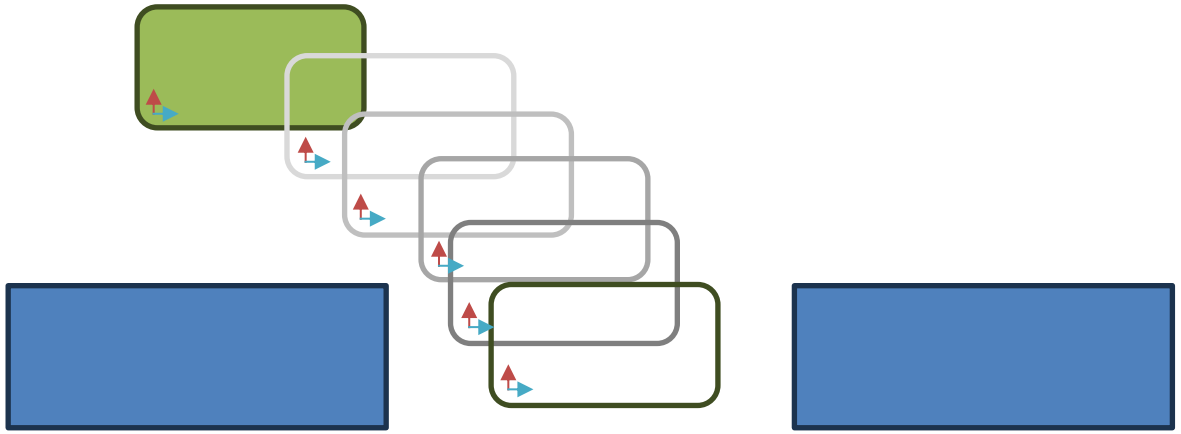
Randomness built into the motion planning method

Partial plan for nearby obstacles or dynamic reactions

Longer view plan to reach final goal

Holonomic

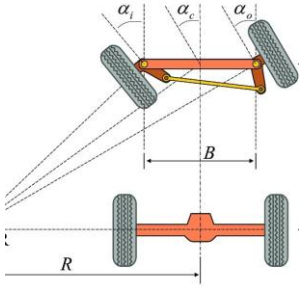
- Can move along any vector within dimensionality of robot
- 2D example



Holonomic – achieved with omniwheels

Orientation and movement direction can be independent

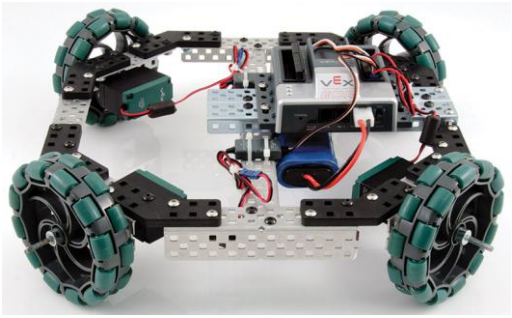
Non-holonomic – traditional steering



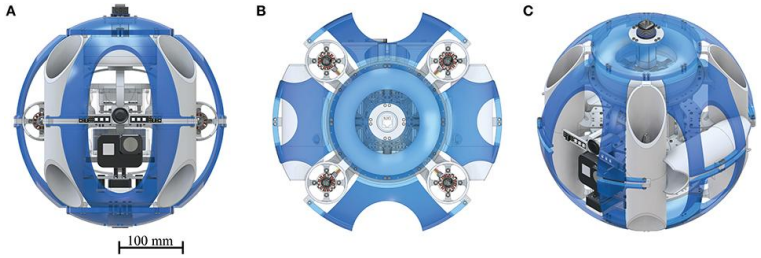
Orientation and movement direction are linked

Examples

- Holonomic



Generic onmi wheel ground robot



SAUV (almost holonomic)
https://doi.org/10.3389/fnbot.2020.00008

Holonomic drone

https://doi.org/10.2514/6.2018-2985

Additional actuation required for holonomic motion

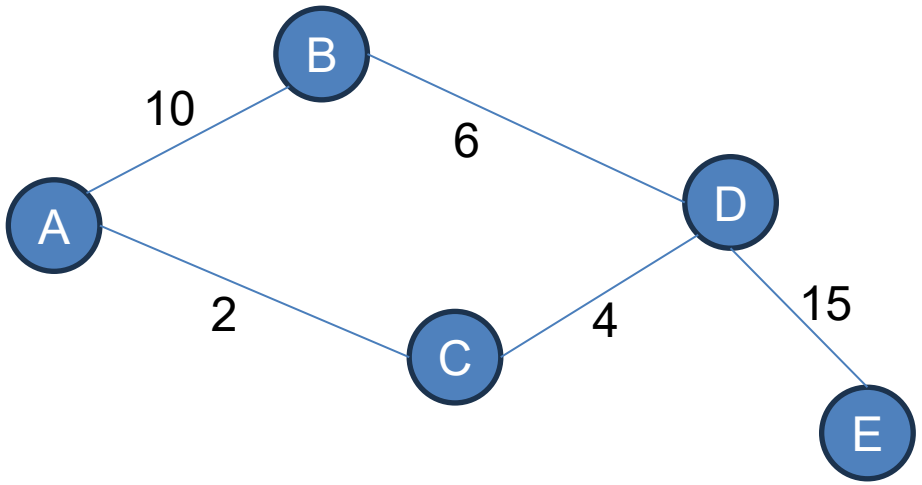
- Non-holonomic

Leo-rover

Multi-copter drones

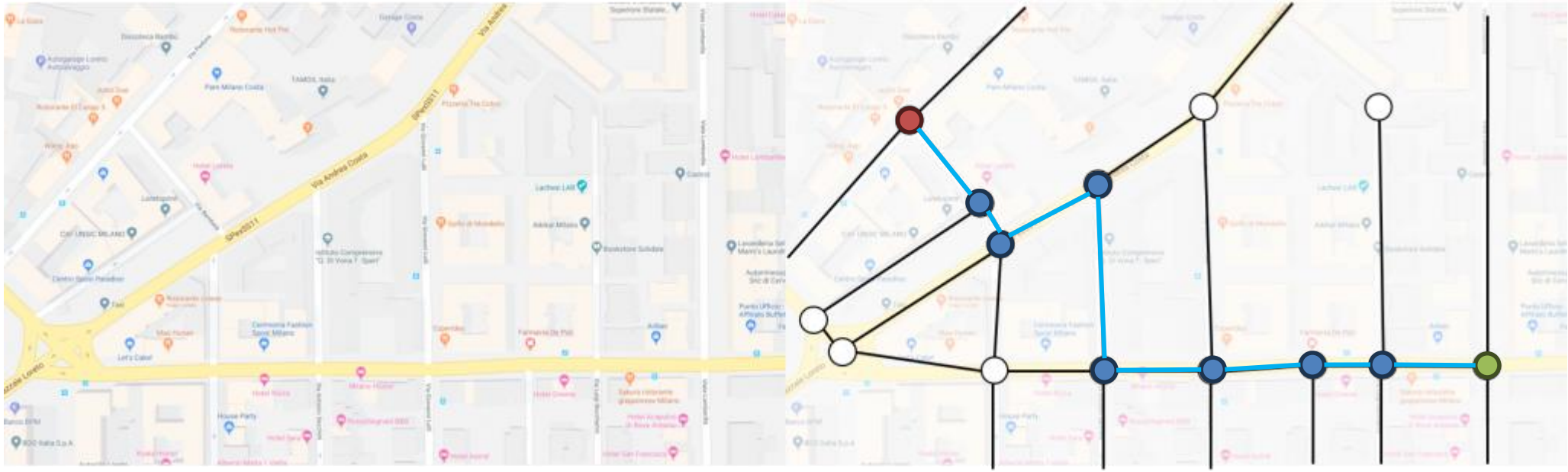
Introduction to graph theory

- What are nodes and edges
  - Nodes – states the system can exist in
  - Edges – available transitions between states
  - Edges can have cost values associated



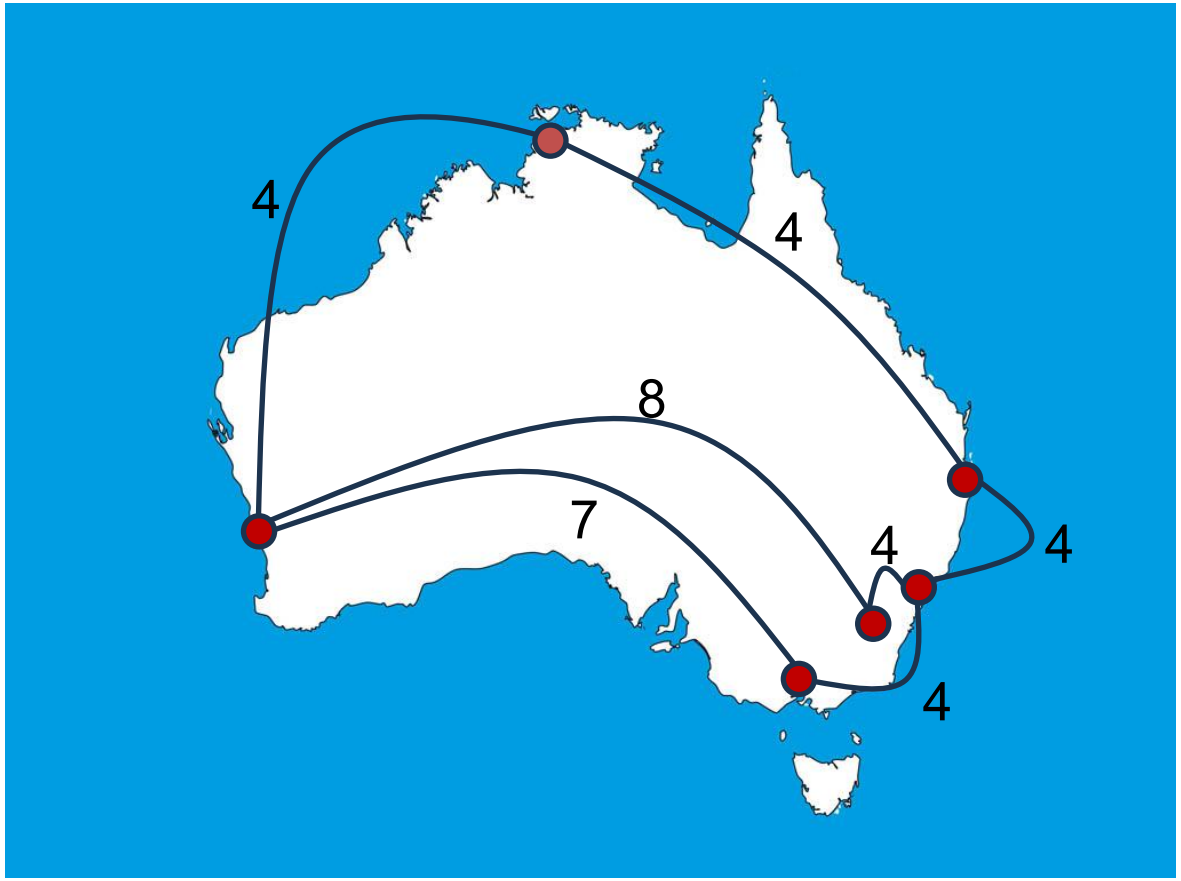
Graph example

- Everyday in satellite navigation
- Nodes are positions between which car moves
- Edges are the available connections
- A list of ordered nodes is a path



Graph theory usages

- Airline scheduling
- Nodes are airports
- Edges are weighted by
- Number of crew required to operate flight
- Used to solve minimum number of crew required without them ‘accumulating’ at any node

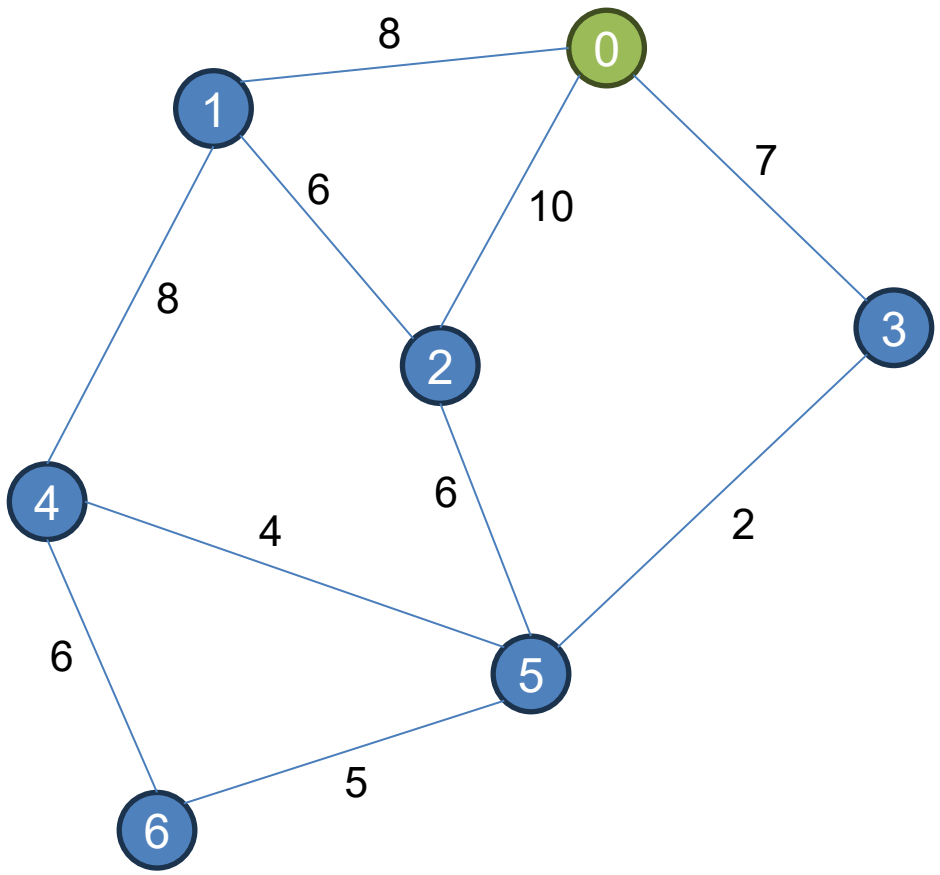


Dijkstra's algorithm

- Find shortest path between a *source node* and all other nodes in the graph
- Assign edges an associated cost
  - (e.g. distances for route planning)
- Edsger W. Dijkstra in 1956 "A note on two problems in connexion with graphs". https://doi.org/10.1007/BF01386390 just 3 pages!

Dijkstra's algorithm

- Start with weighted graph
- Find the shortest path from node '0' to all other nodes in the graph
- Make an initial list of distances and a list of unvisited nodes

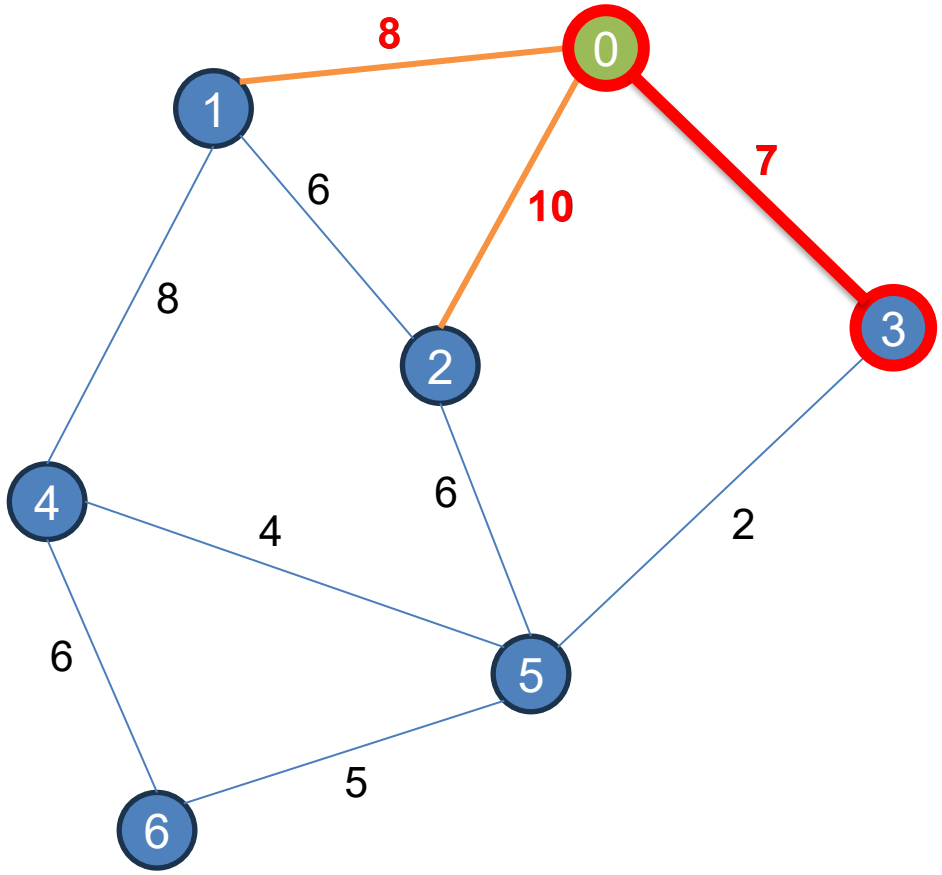


Distance to node		Unvisited
0 :	0	~~0~~
1 :	??	1
2 :	??	2
3 :	??	3
4 :	??	4
5 :	??	5
6 :	??	6

Dijkstra's algorithm

- Use the graph to update distance from start node '0' to all adjacent nodes
- Does not mean shortest path is found yet.
- Select node that is closest, and mark as visited.
- Update the current 'path' as shortest

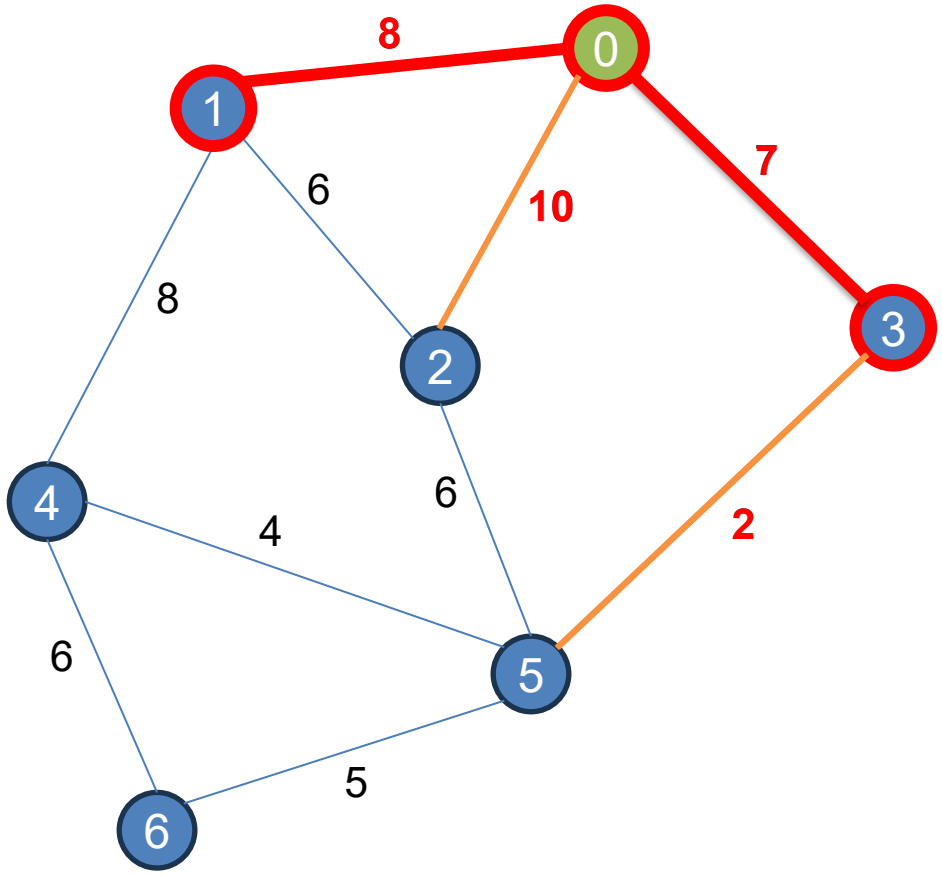
Distance to node		Unvisited
0 :	0	~~0~~
1 :	~~??~~ 8	1
2 :	~~??~~ 10	2
3 :	~~??~~ 7	~~3~~
4 :	??	4
5 :	??	5
6 :	??	6



= "current path"

Dijkstra's algorithm

- Find distances from all nodes in current path to all adjacent nodes
- Nodes in path [0,3], adjacent nodes [1,2,5]
- From the unvisited nodes, select the shortest path and mark as visited.

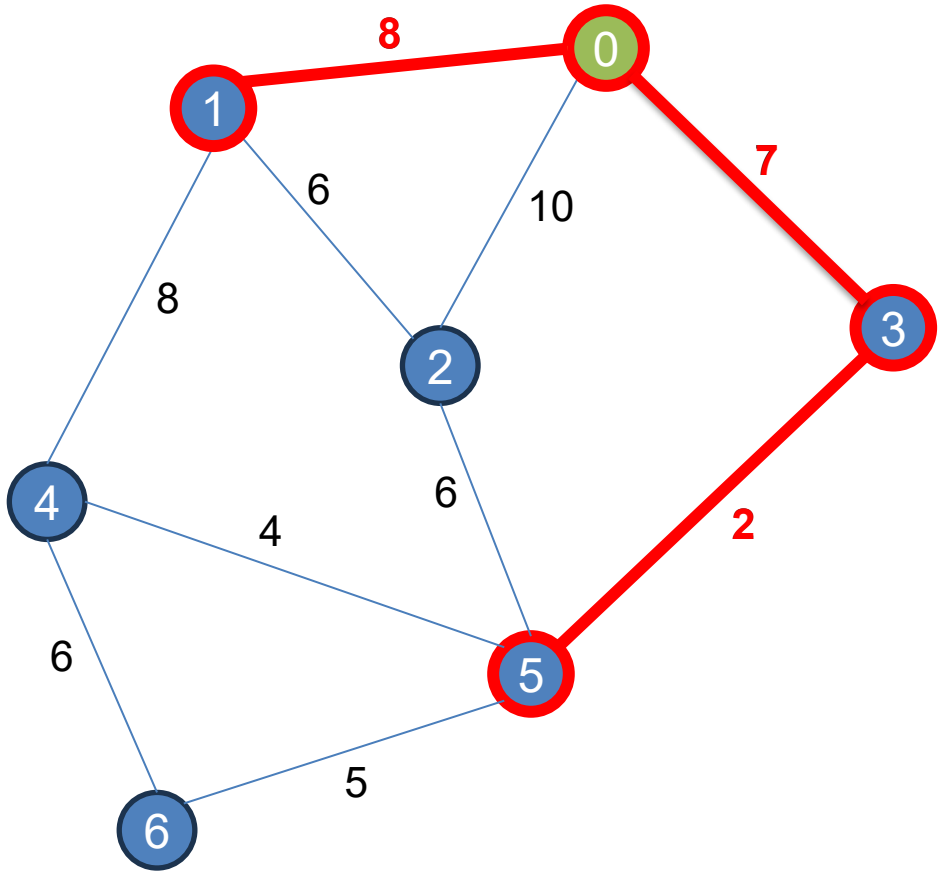


Distance to node		Unvisited
0 :	0	~~0~~
1 :	8	~~1~~
2 :	10	2
3 :	7	~~3~~
4 :	??	4
5 :	7+2 = 9	5
6 :	??	6

Dijkstra's algorithm

- Repeat again.
- Nodes in path [0,1,3] adjacent nodes [2,4,5]
  - 0 to 2 = 10 or 8+6 (choose smallest)
  - 0 to 4 = 8+8
  - 0 to 5 = 7+2
- Choose shortest from unvisited

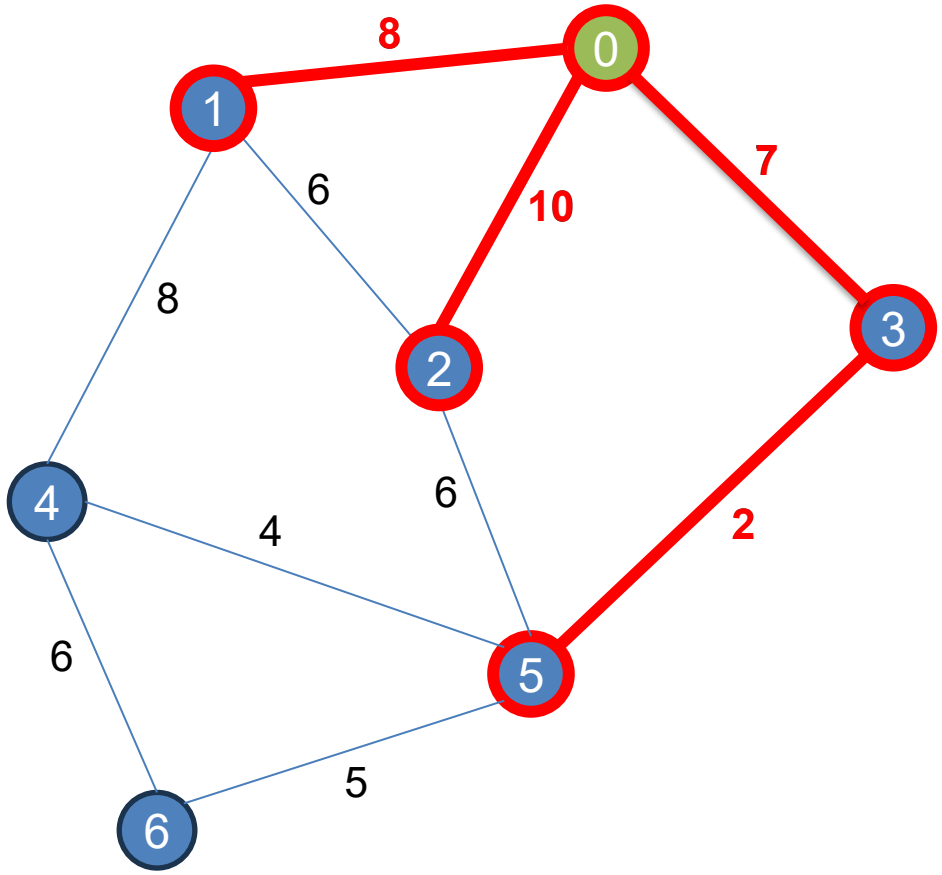
Distance to node		Unvisited
0 :	0	~~0~~
1 :	8	~~1~~
2 :	10	2
3 :	7	~~3~~
4 :	16	4
5 :	9	~~5~~
6 :	??	6



Dijkstra's algorithm

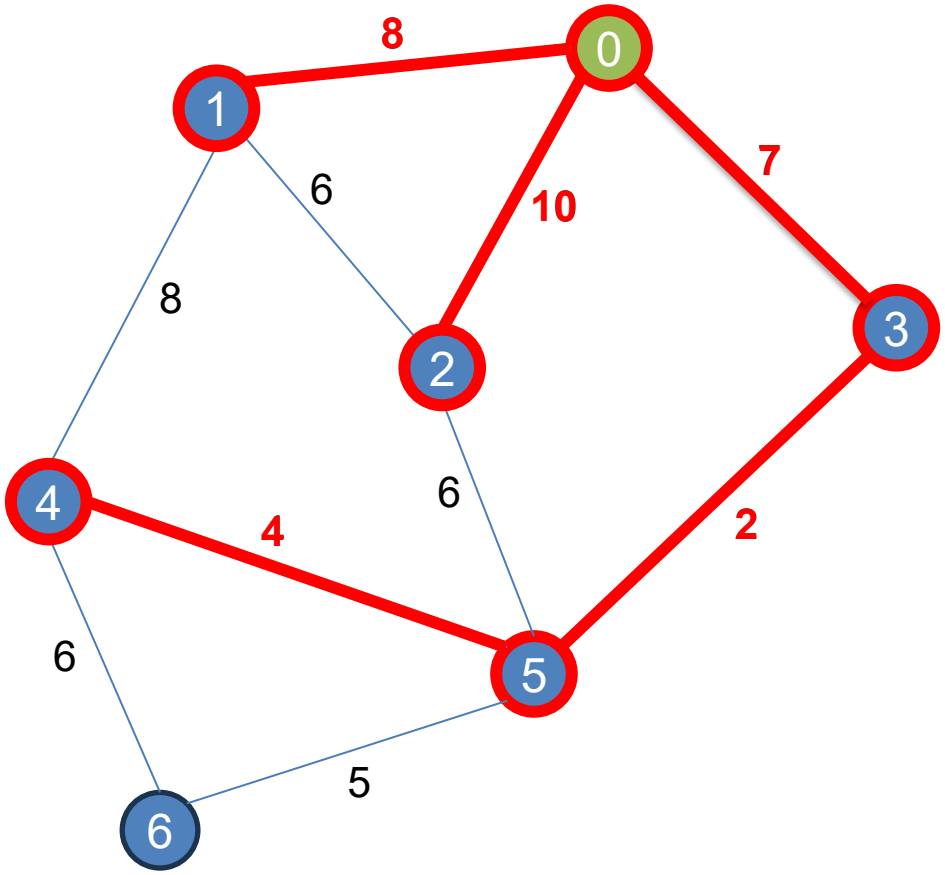
- And again…
- Nodes in path [0,1,3,5] adjacent nodes [2,4,6]
  - 0 to 2 = 10 or 8+6 or 7+2+6 (pick shortest)
  - 0 to 4 = 8+8 or 7+2+4 (pick shortest)
  - 0 to 6 = 7+2+5
- Choose shortest from unvisited

Distance to node		Unvisited
0 :	0	~~0~~
1 :	8	~~1~~
2 :	10	~~2~~
3 :	7	~~3~~
4 :	13	4
5 :	9	~~5~~
6 :	14	6



Dijkstra's algorithm

- And again…
- Nodes in path [0,1,2,3,5] adjacent nodes [4,6]
  - 0 to 4 = 8+8 or 7+2+4 (pick shortest)
  - 0 to 6 = 7+2+5
- Choose shortest from unvisited

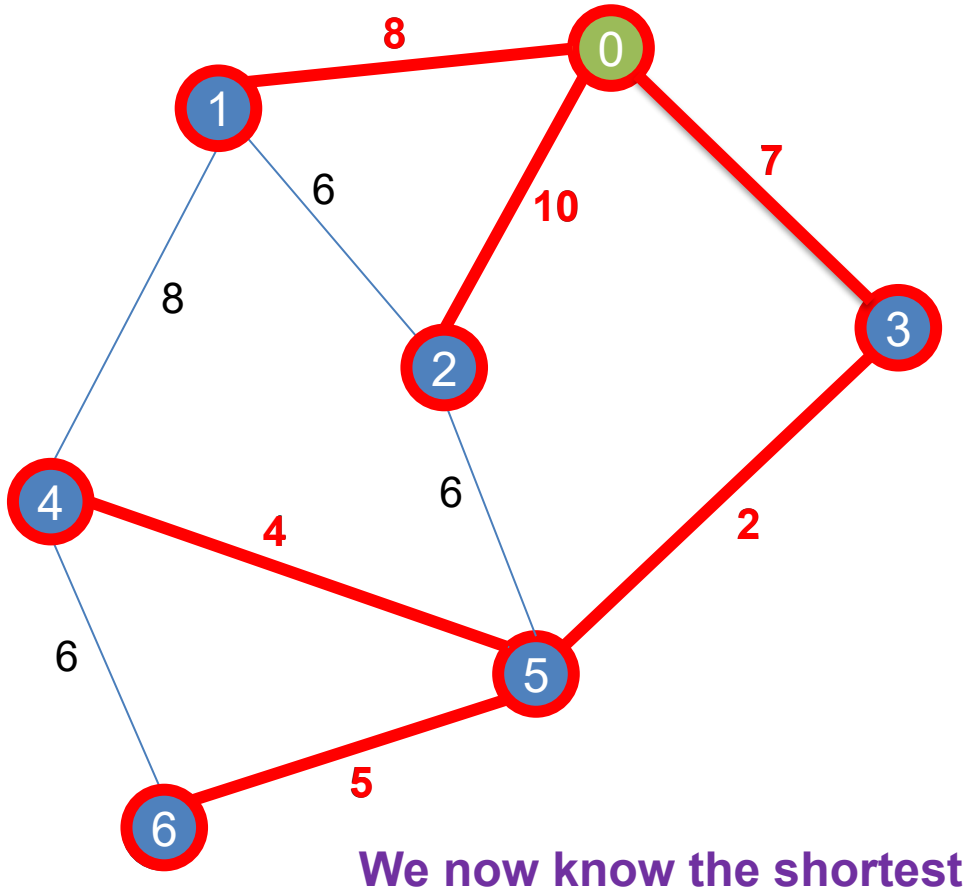


Distance to node		Unvisited
0 :	0	~~0~~
1 :	8	~~1~~
2 :	10	~~2~~
3 :	7	~~3~~
4 :	13	~~4~~
5 :	9	~~5~~
6 :	14	6

Dijkstra's algorithm

- Finally
- Nodes in path [0,1,2,3,4,5] adjacent nodes [6]
  - 0 to 6 = 7+2+5 or 7+2+4+6
- Choose shortest from unvisited
- All nodes marked as visited – algorithm is complete
- Shortest path is the sub-graph marked in red

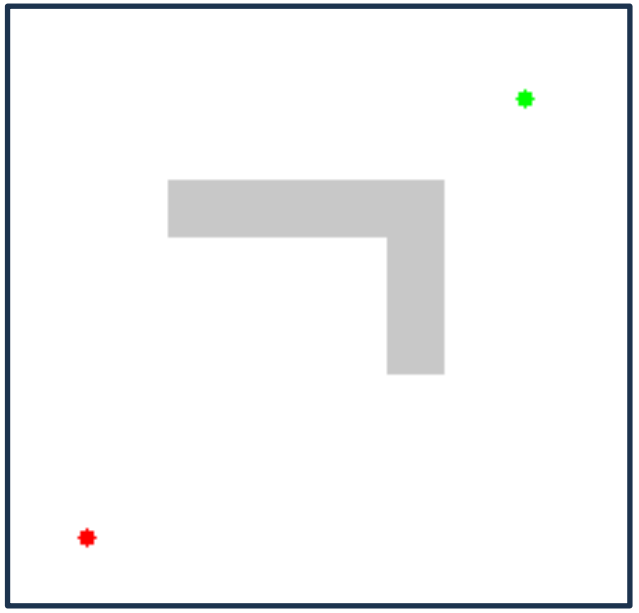
Distance to node		Unvisited
0 :	0	~~0~~
1 :	8	~~1~~
2 :	10	~~2~~
3 :	7	~~3~~
4 :	13	~~4~~
5 :	9	~~5~~
6 :	14	~~6~~



We now know the shortest path across the graph between the start and any other node.

Dijkstra Cartesian motion planning

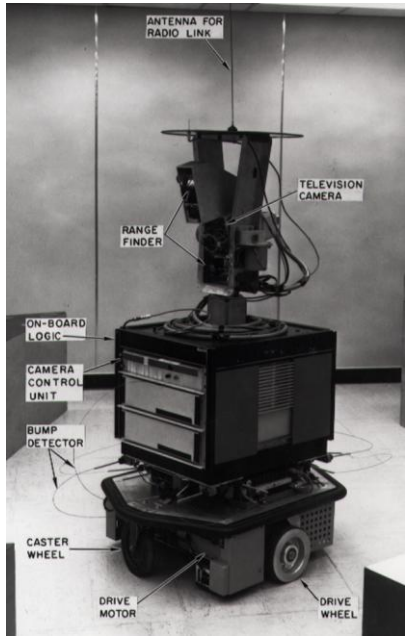
- What if all nodes are regularly spaced with equal weights
- Directions aligned to Cartesian axes
- Connectivity defined by position in the grid
- Cover entire area/volume with nodes
- Obstacles represented by missing nodes and edges



Dijkstra algorithm challenges

- Inefficient – computation can be very long
  - Breadth-first search
  - Scales with the number of edges and vertices $O(|E| + |V|^2)$
  - Some savings with implementations and sparse graphs
- Guarantees shortest paths - optimal
- In robotics we often only want to find shortest distance between start and goal
- The A* algorithm is a goal directed variant of Dijkstra's

A* (A-star) algorithm



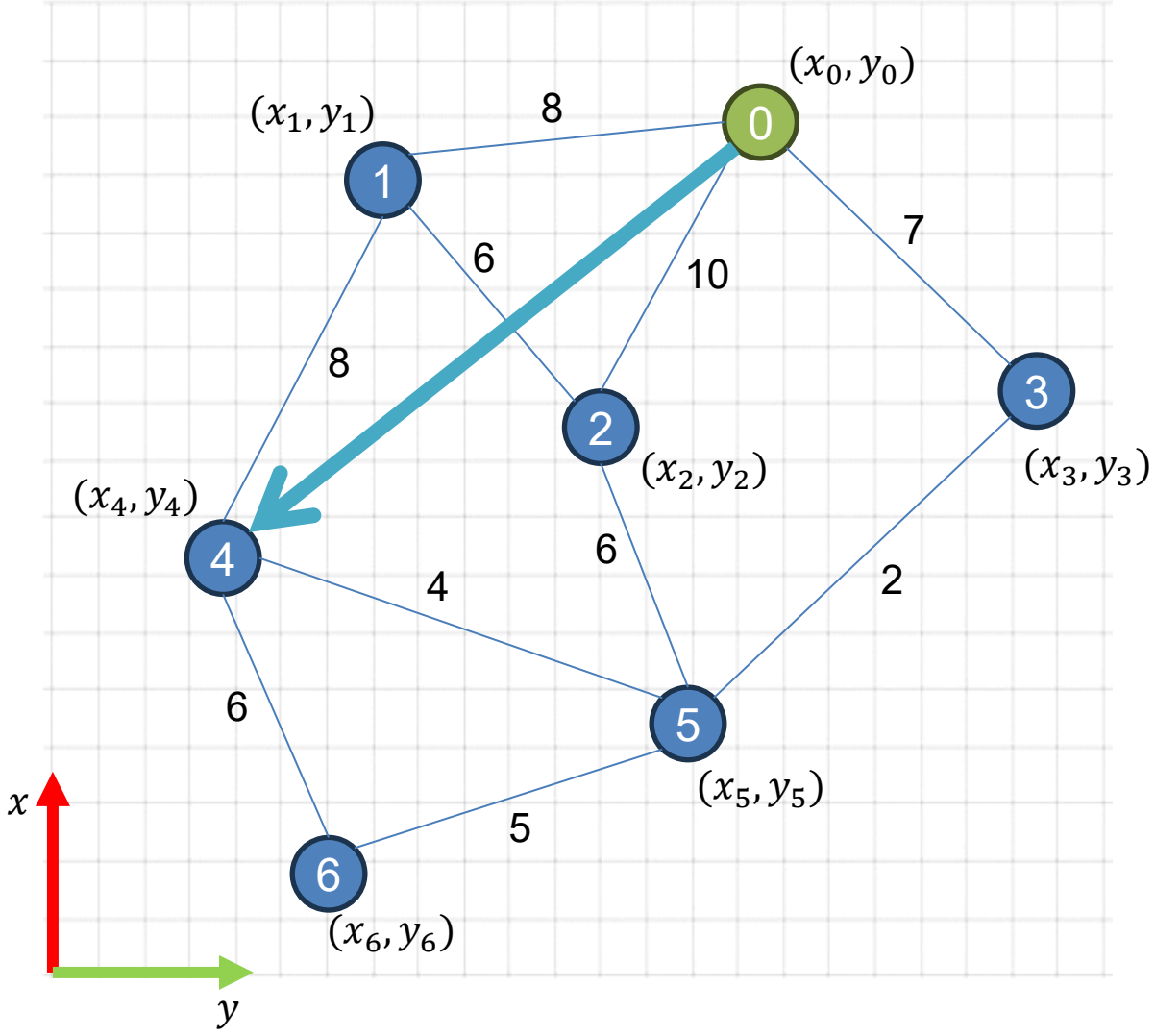
- Created as part of the Shakey project 1968.
  - Hart, Nilsson, Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths” https://doi.org/10.1109/TSSC.1968.300136
- Combination of Dijkstra and Greedy best-first search
  - Informed search algorithm
- Goal-orientated to direct search to goal node rather than all
- Introduce a **heuristic**
  - often Euclidian distance is used in Cartesian frame
- Remains optimal if heuristic modeling is valid
- For mobile robots, the graph is an **occupancy grid**

Heuristic

- Nodes and edges extended with additional information.
- If coordinates of nodes are known, then Euclidian distances can be calculated as a heuristic
- e.g. distance from 0 to 4

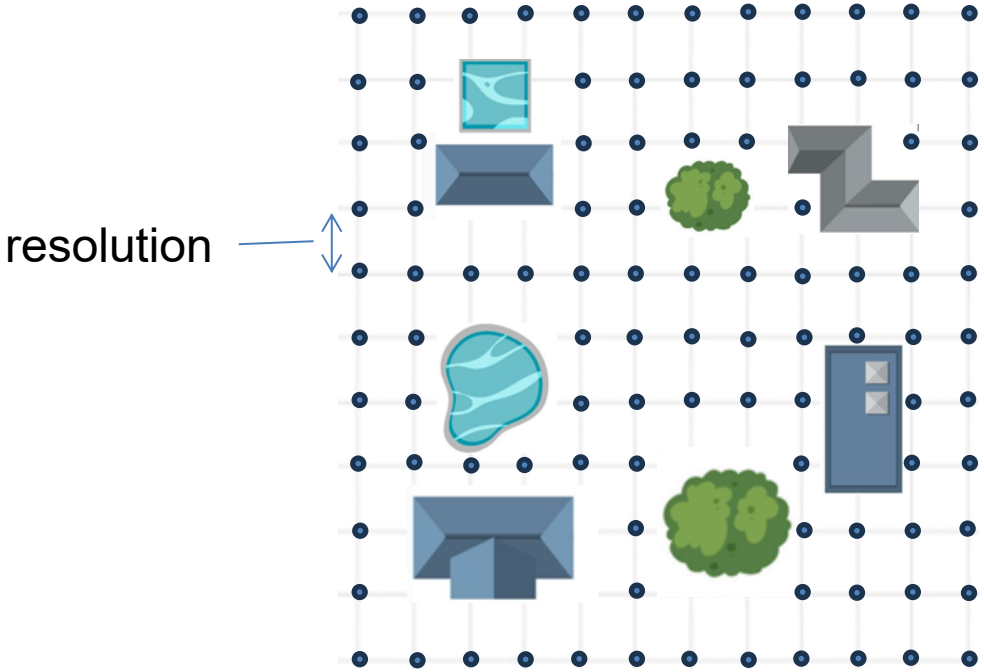
$$h(n) = \left\| \begin{pmatrix} x_4 \\ y_4 \end{pmatrix} - \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} \right\|$$

- Heuristic is not necessarily related to the edge weightings



Occupancy grid

- Mobile robots assumed to have free motion over a continuum.
- Apply grid of regularly spaced nodes over the area using a reference frame. Nodes only connect to nearest neighbours.
- Assume all edge weightings are equal.
- Remove nodes for areas that cannot be visited (obstacles)
- For simplicity, represent as an array with 1's and 0's and known spacing resolution.



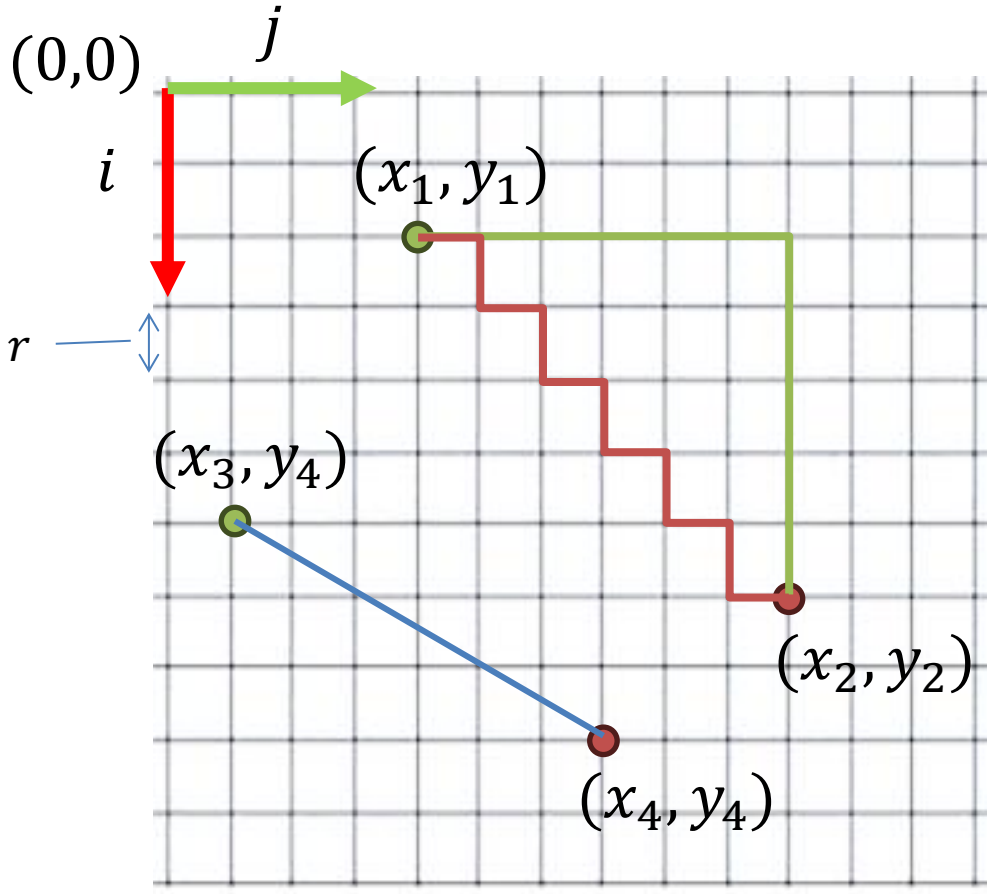
$$grid = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Heuristics on occupancy grid

- With known resolution distance heuristic between nodes can be calculated from location indices

- Manhattan distance
  - Axes treated independently
  - Constrained to edges
  - $h(n) = r[(x_2 - x_1) + (y_2 - y_1)]$

- Euclidean distance
  - Straight line distance
  - $h(n) = r\sqrt{(x_4 - x_3)^2 + (y_4 - y_3)^2}$



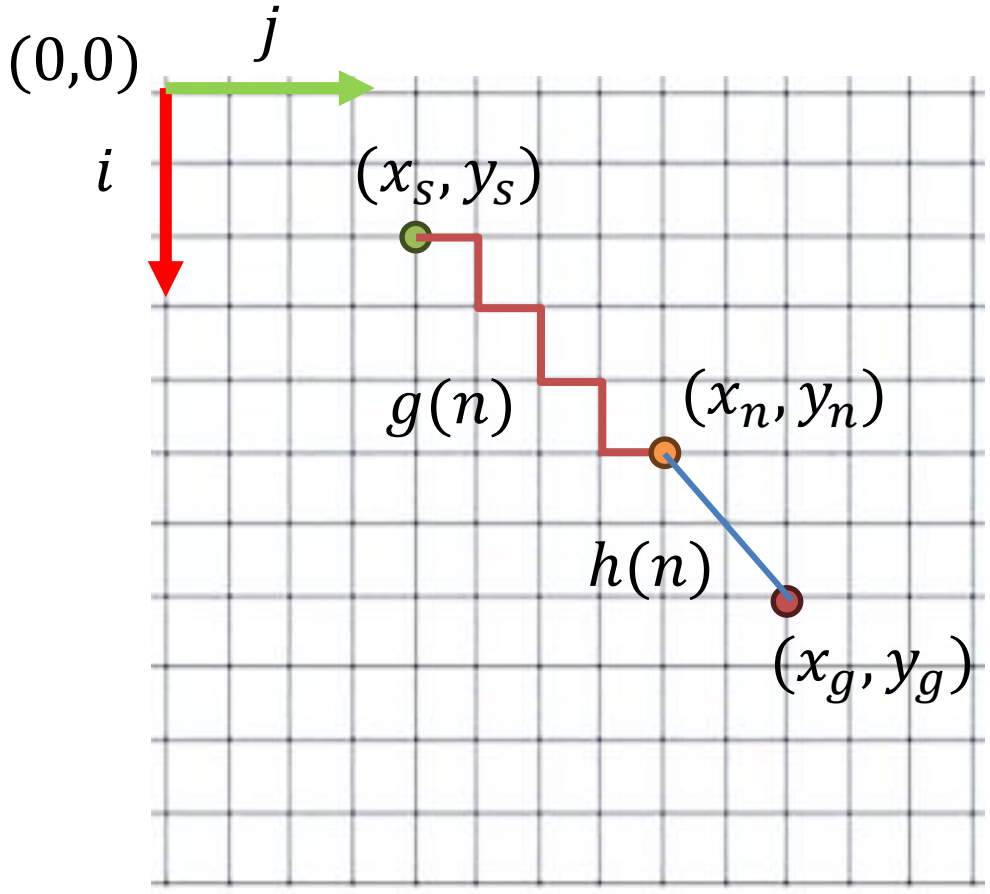
Cost function for node n

Define node n as somewhere along a path being planned

$$f(n) = g(n) + h(n)$$

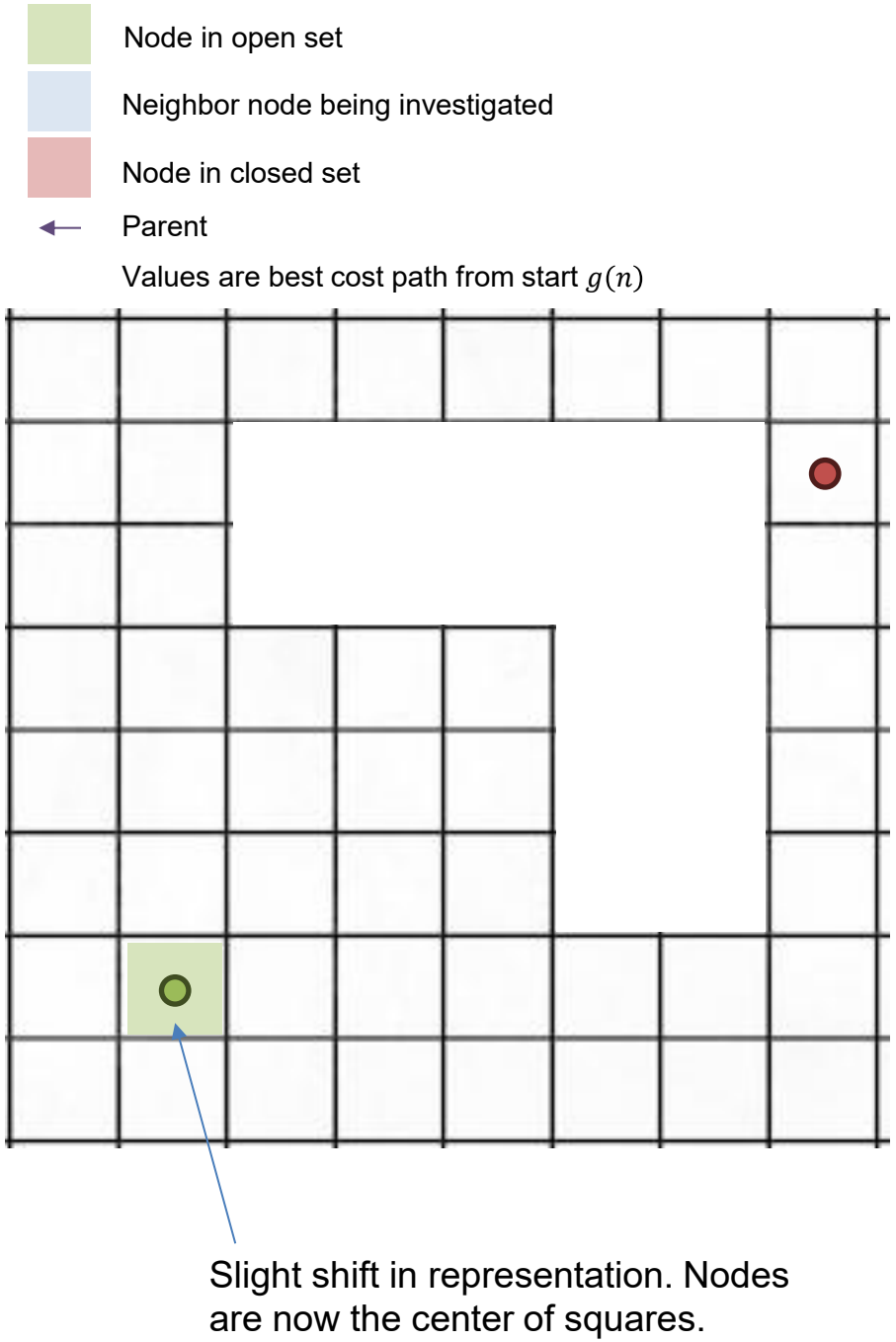
$g(n)$ -- the same as Dijkstra. The cost of the best path from the starting location to the current node being inspected navigating the graph. Must obey occupancy and connectivity of the graph. i.e. cannot cross voids.

$h(n)$ -- new information. Estimated cost from the current node being inspected to the goal. Uses heuristic function. Heuristic does not have to obey connectivity of graph.



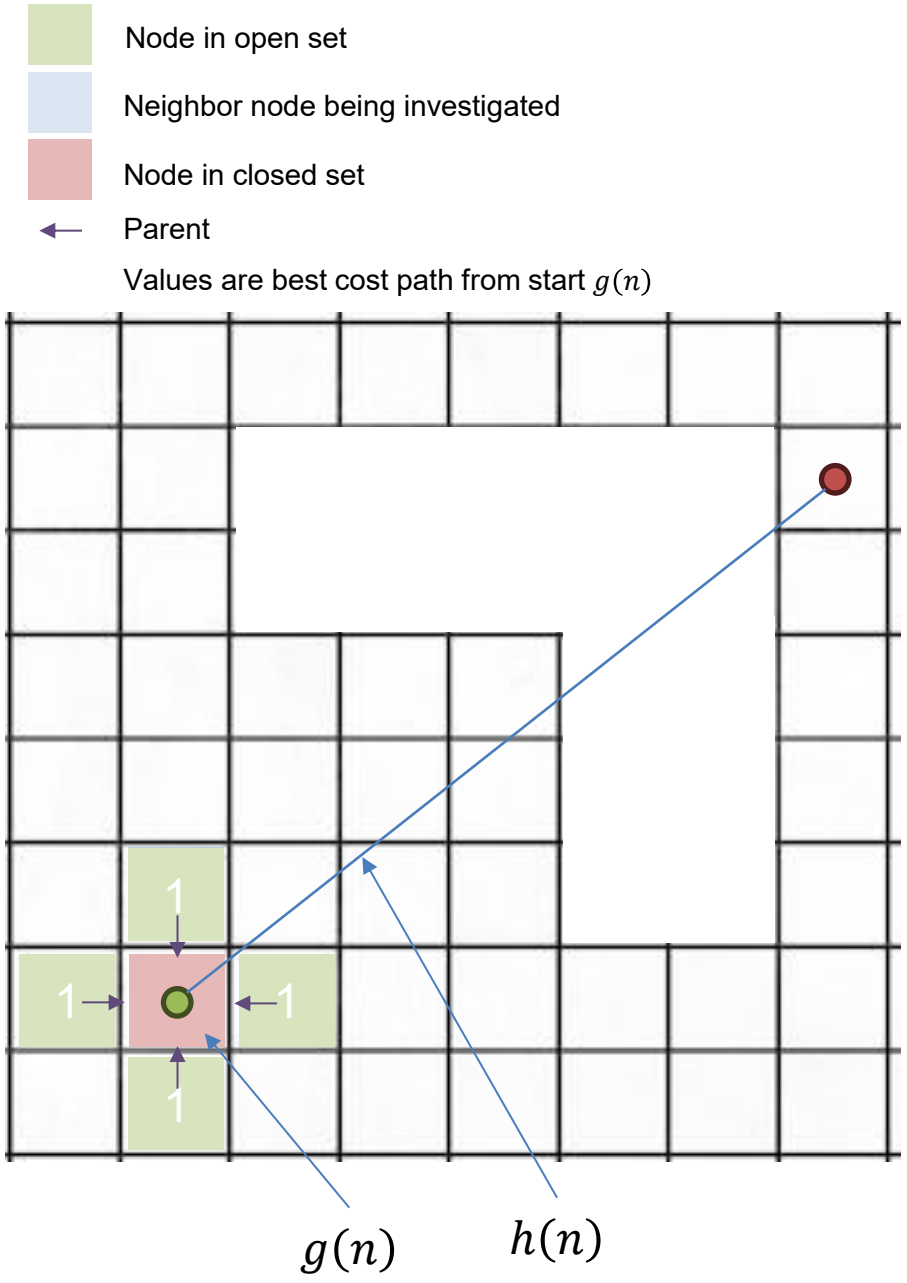
A* Example

- Define occupancy grid, start, and end locations.
- Create list of ‘Open’ nodes – “frontier”.
  - Nodes which can still be ‘explored’
  - Not all neighbors visited yet.
  - Add the starting location only.
- Create a list of ‘Closed’ nodes – “interior”.
  - Nodes with a known cost $f(n)$.
  - All neighbors visited already.
  - Start this list empty.
- Create a list of parent nodes – for each node, there will be a previous node which is on the shortest path.
- Assign best path costs to each node



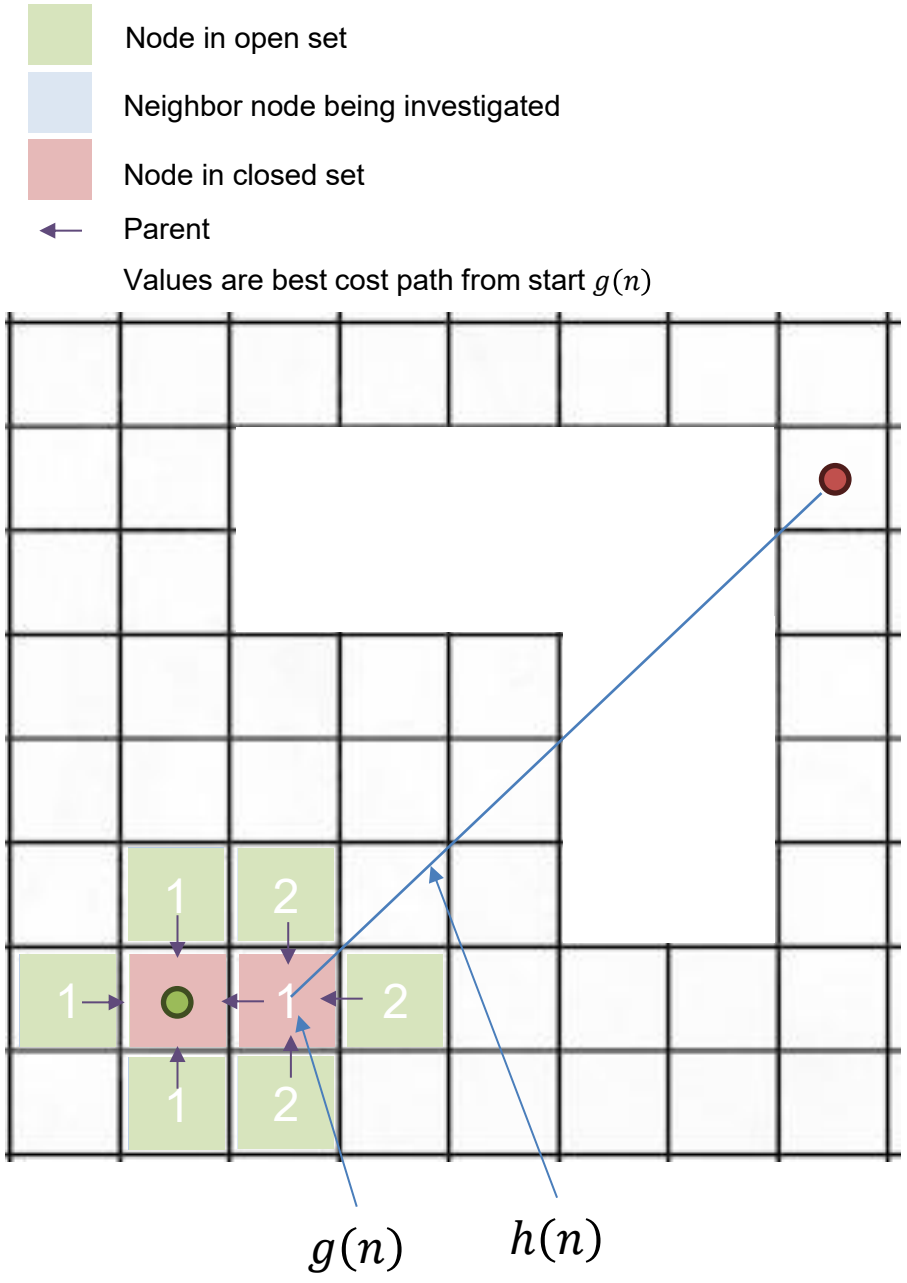
A* Example

- For every node in the Open set, calculate $f(n)$
  - Costs to move ($g(n)$) can be calculated from the Manhattan distance.
  - Cost to goal ($h(n)$) can be calculated from the Euclidian distance.
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



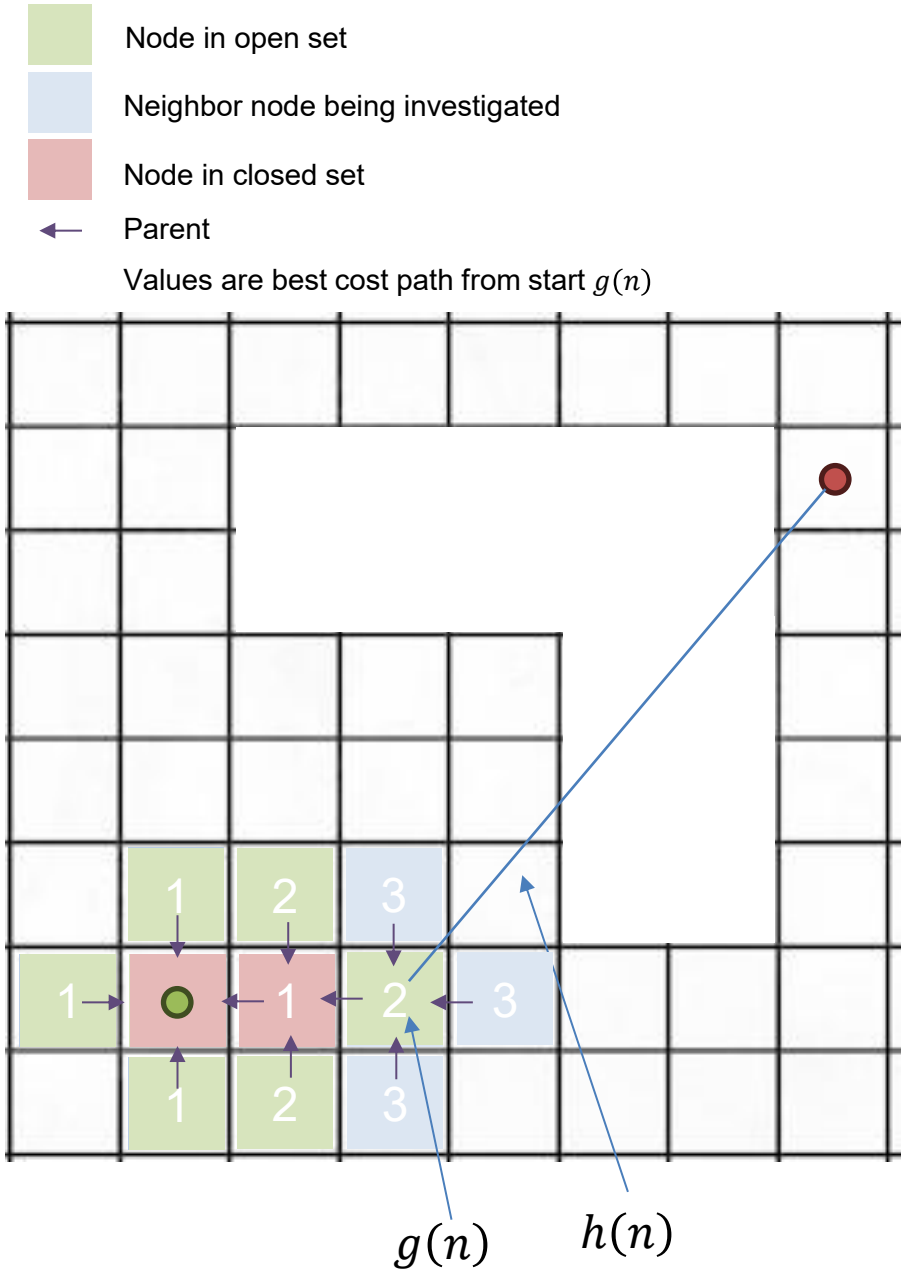
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



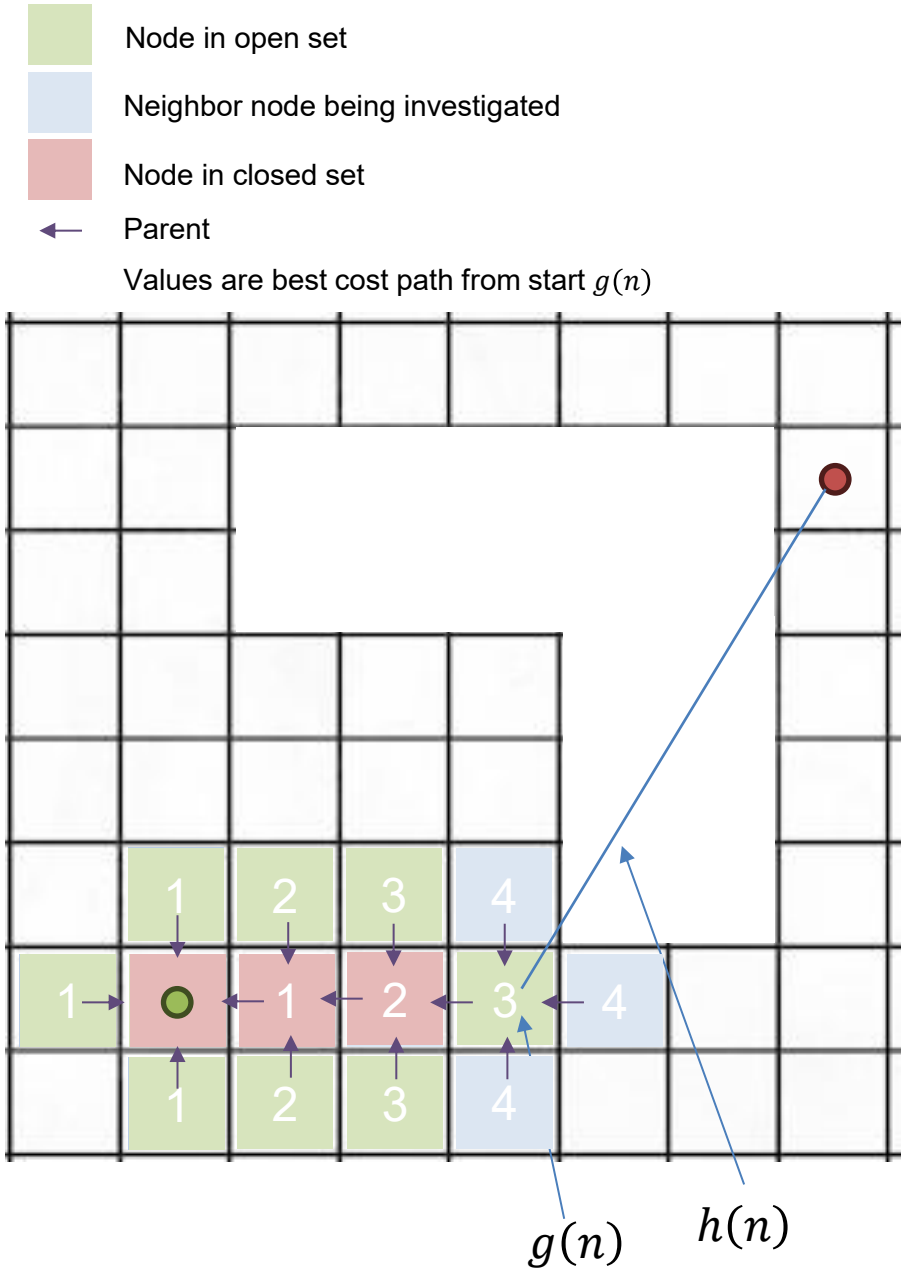
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



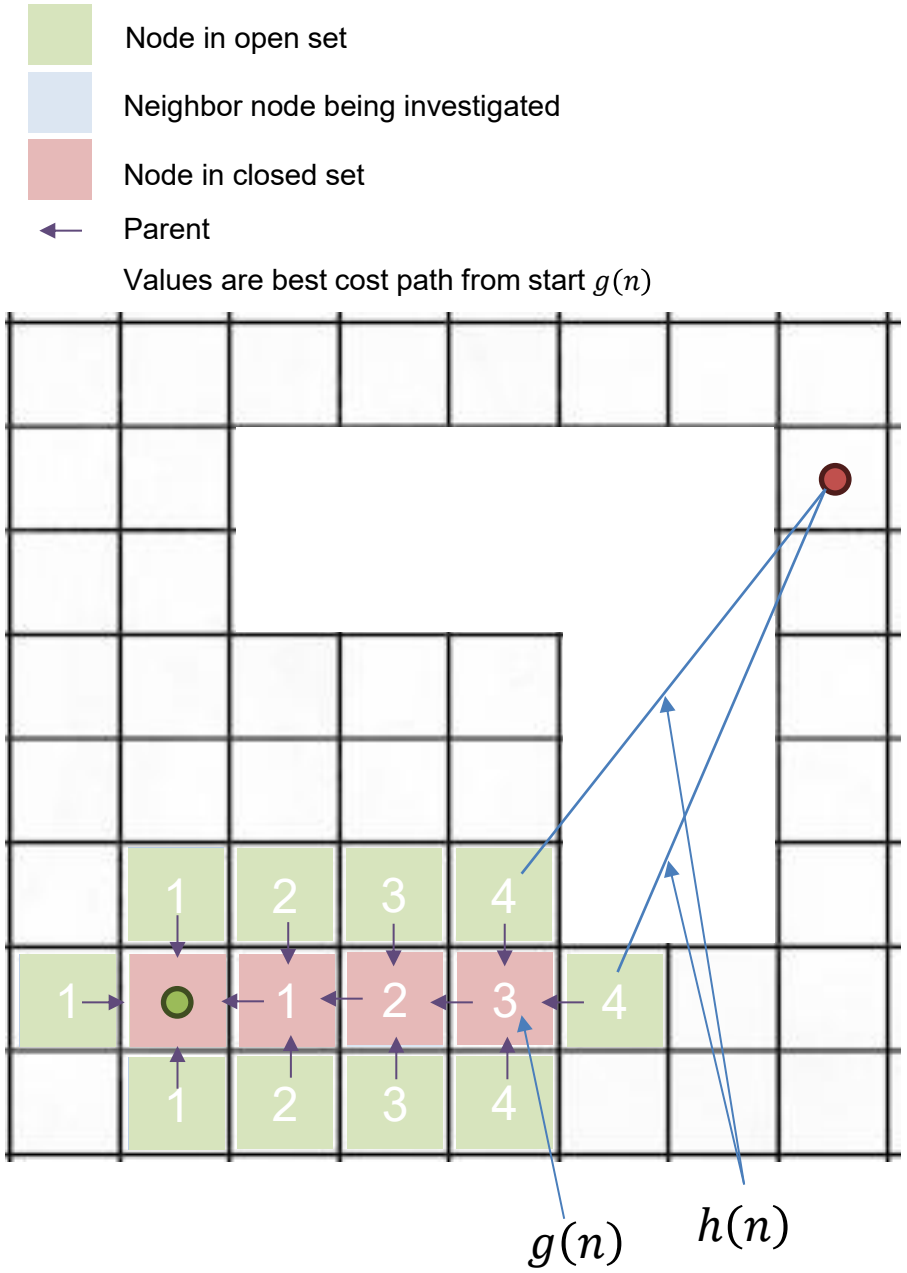
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



A* Example

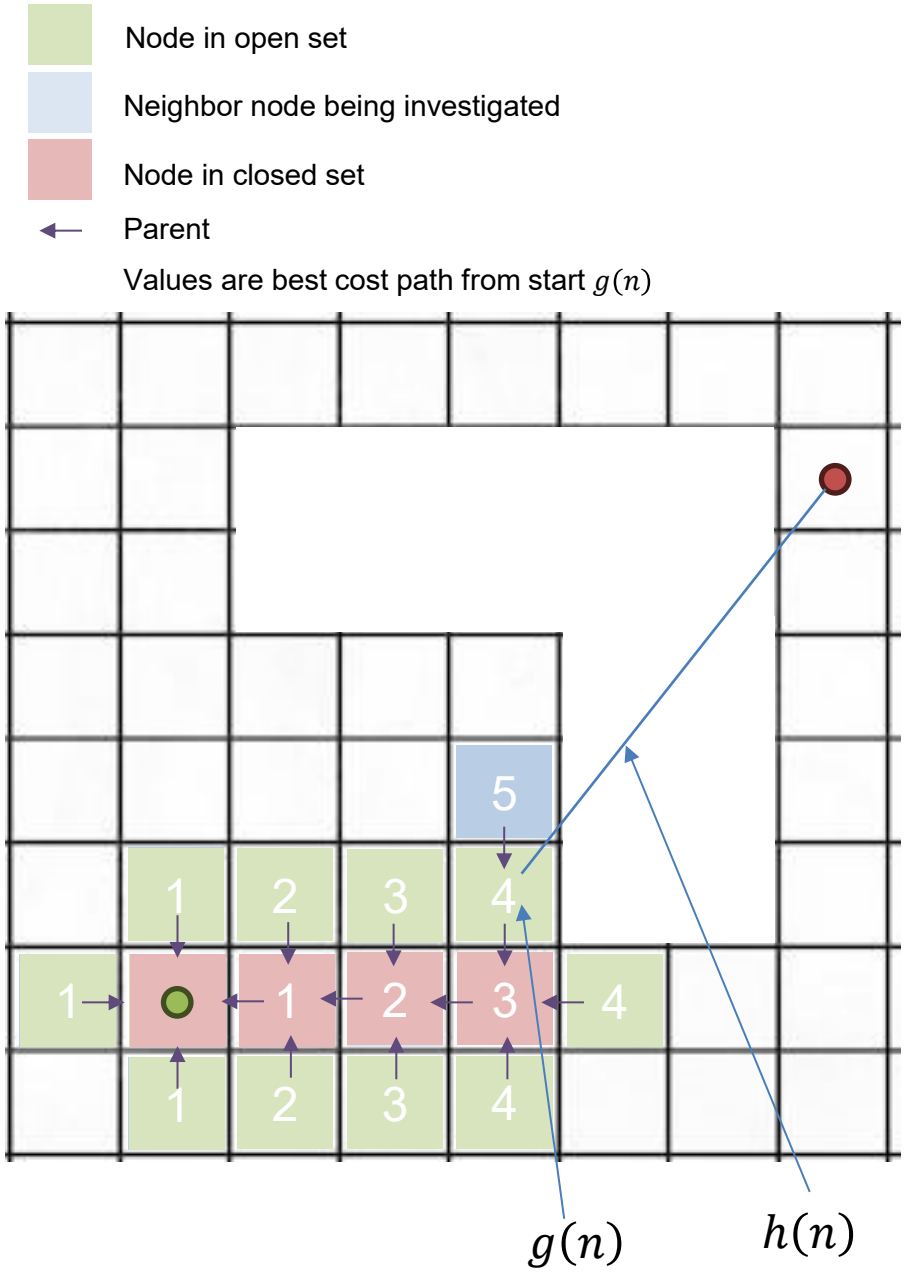
- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



Lowest cost determined by heuristic from the available Open set options

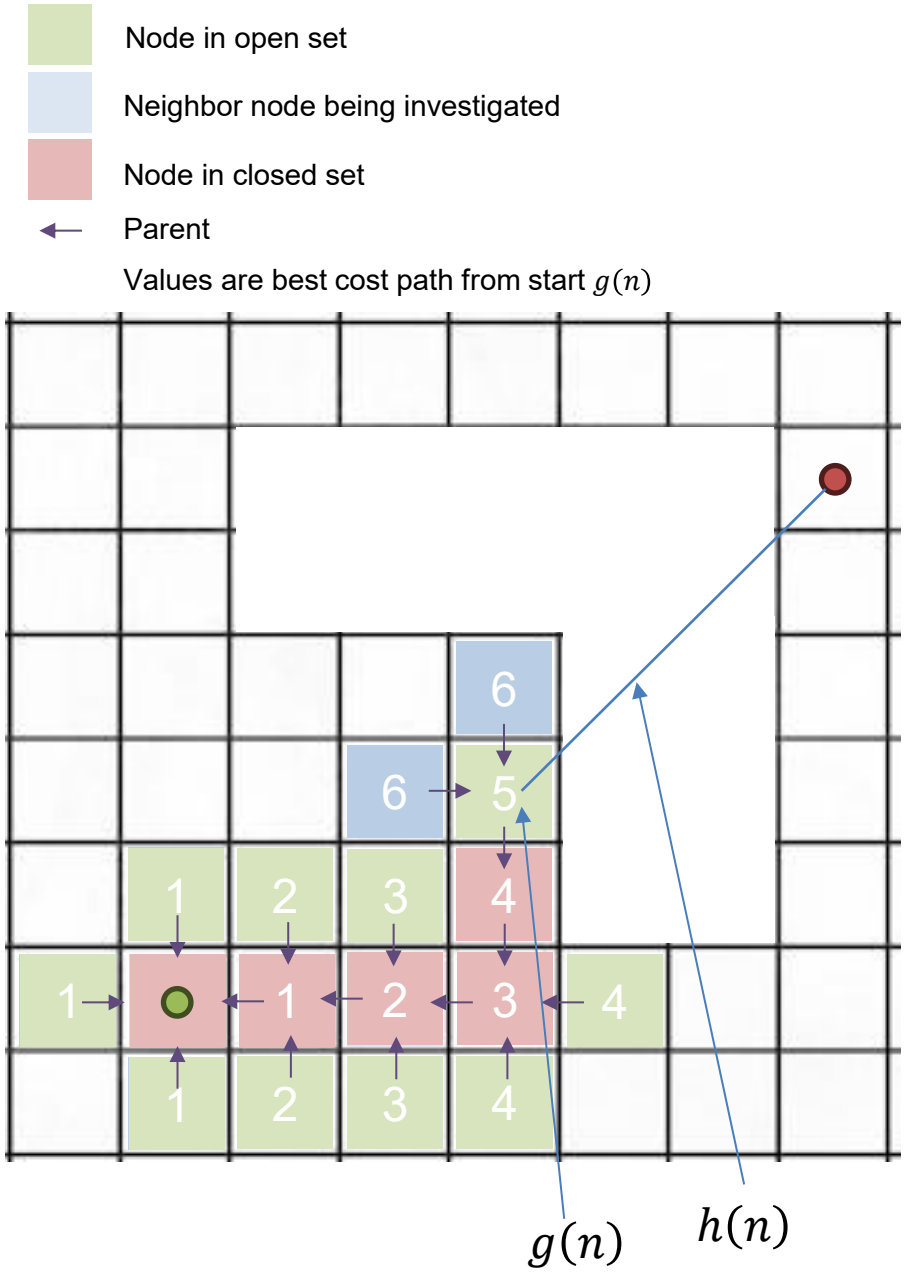
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



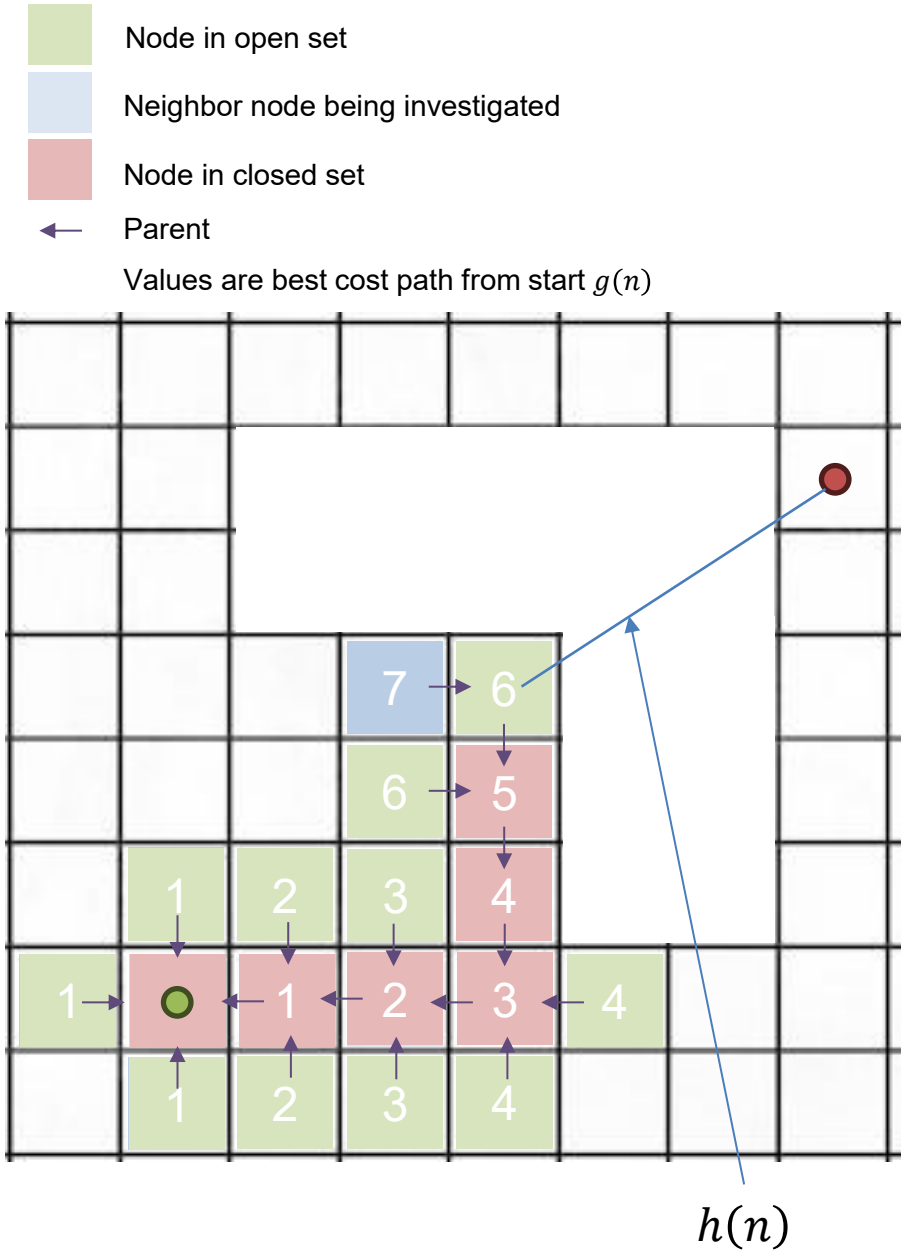
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



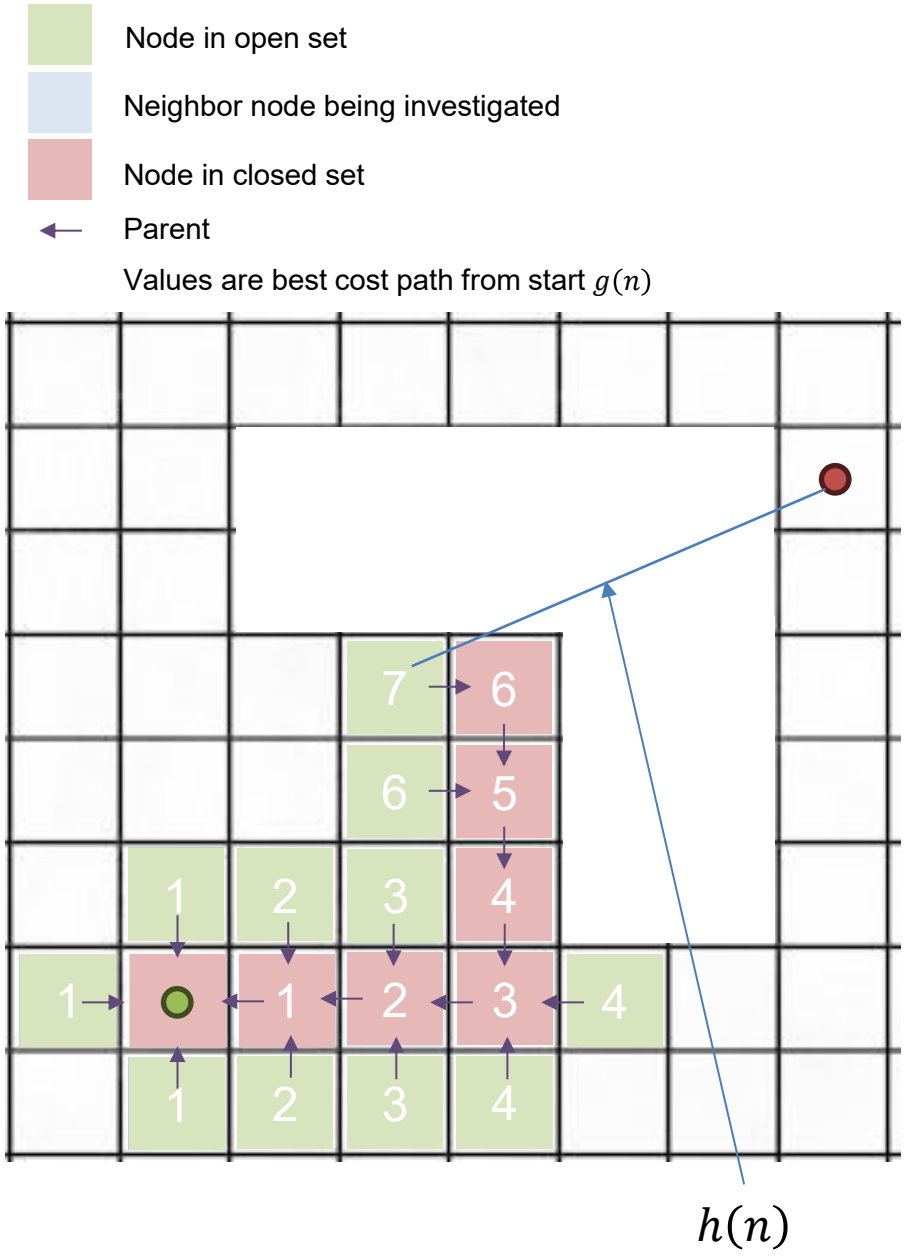
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



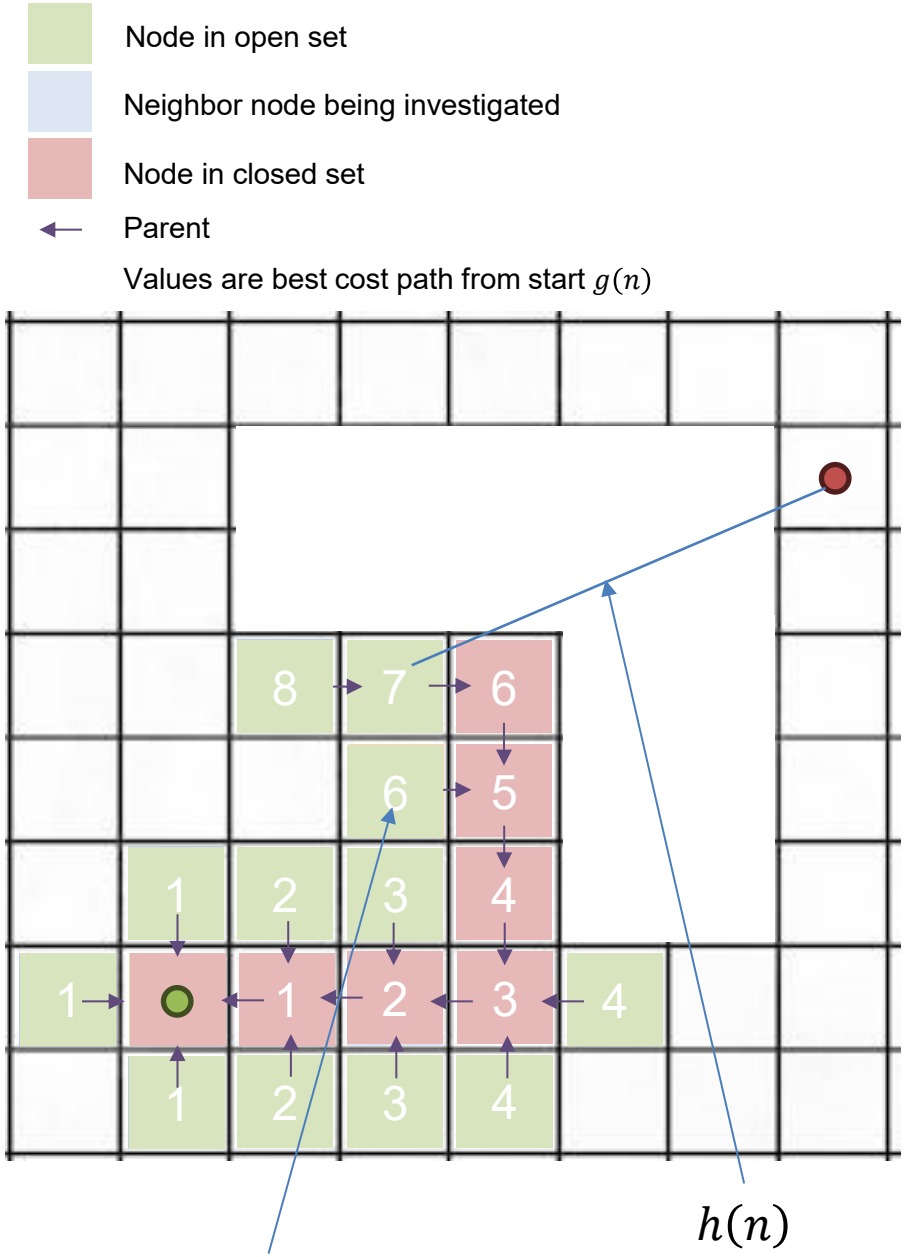
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



A* Example

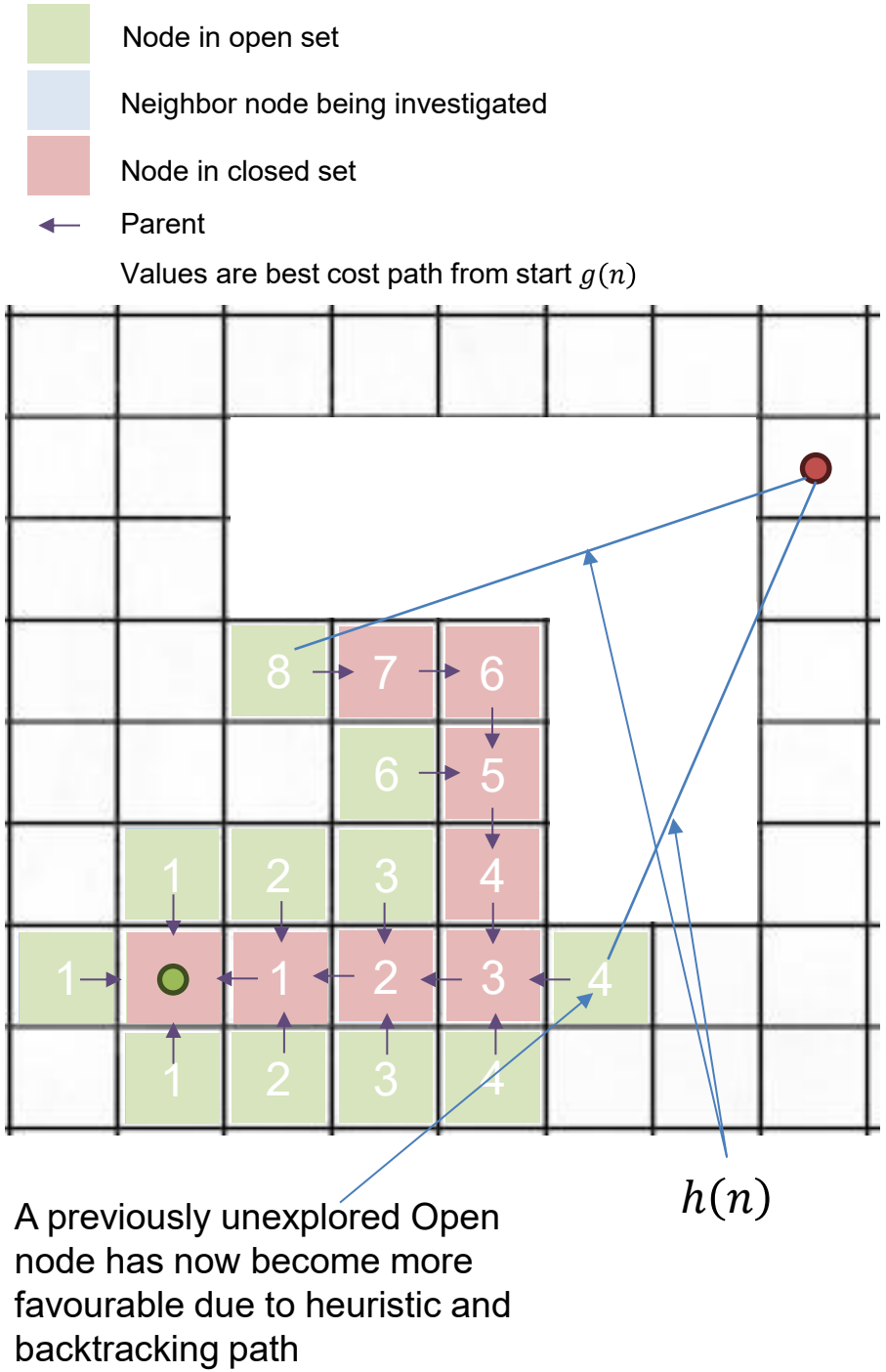
- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



Path via parent nodes would be 8, but this node already has a shorter path. So do not change.

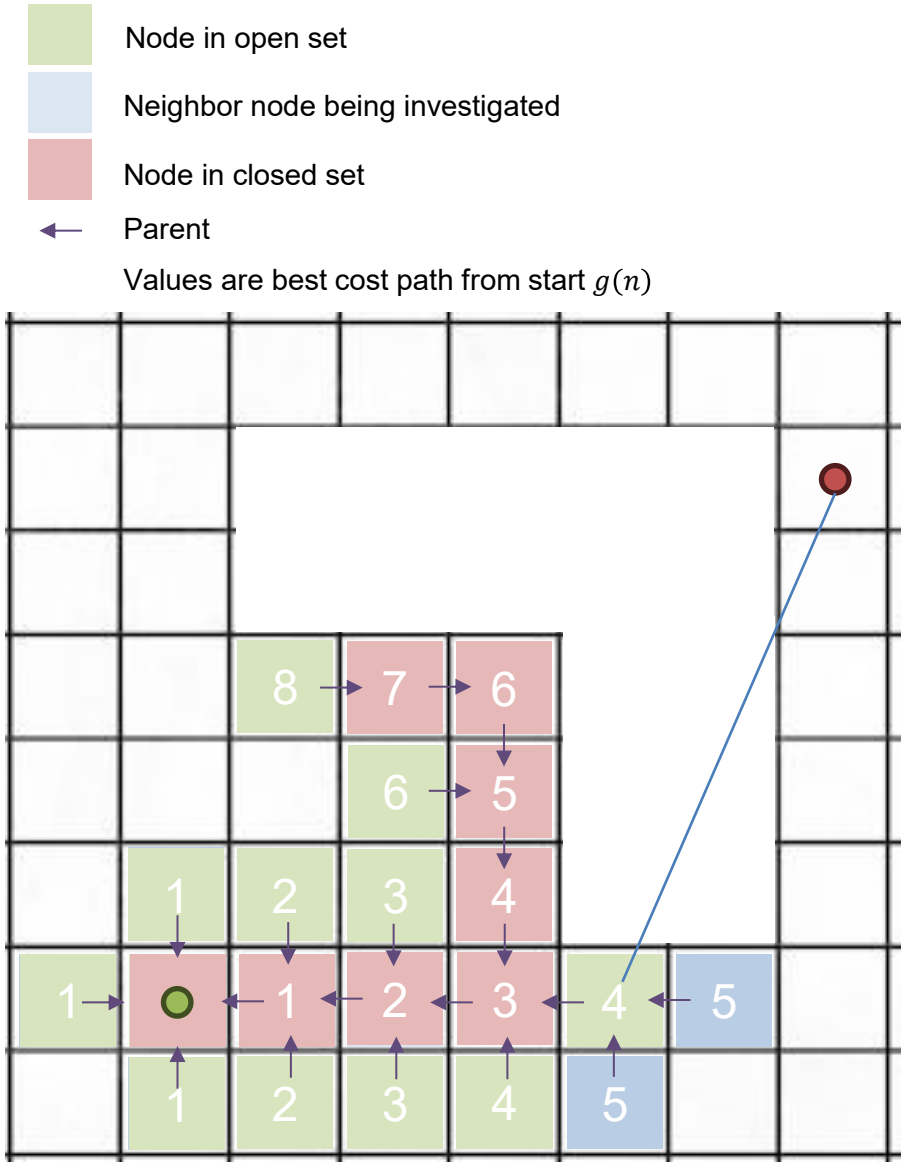
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



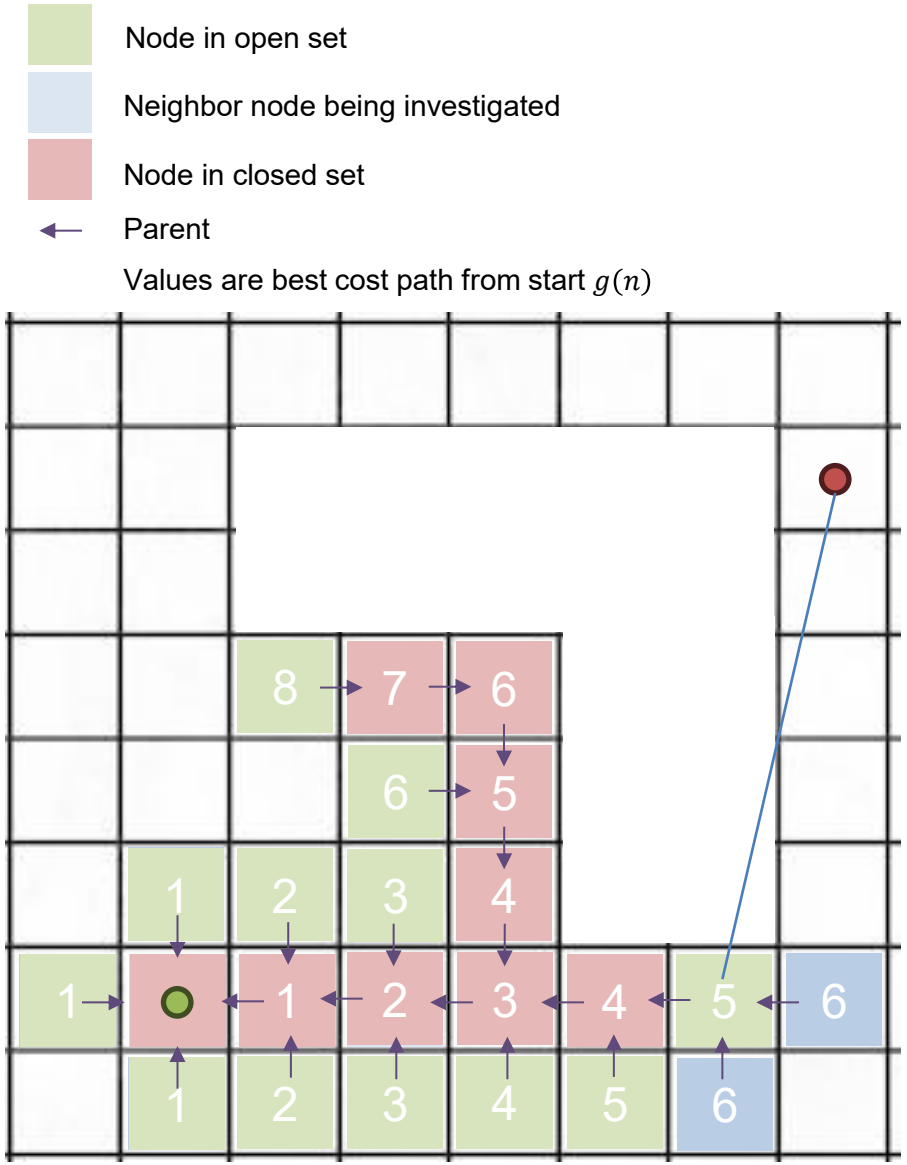
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



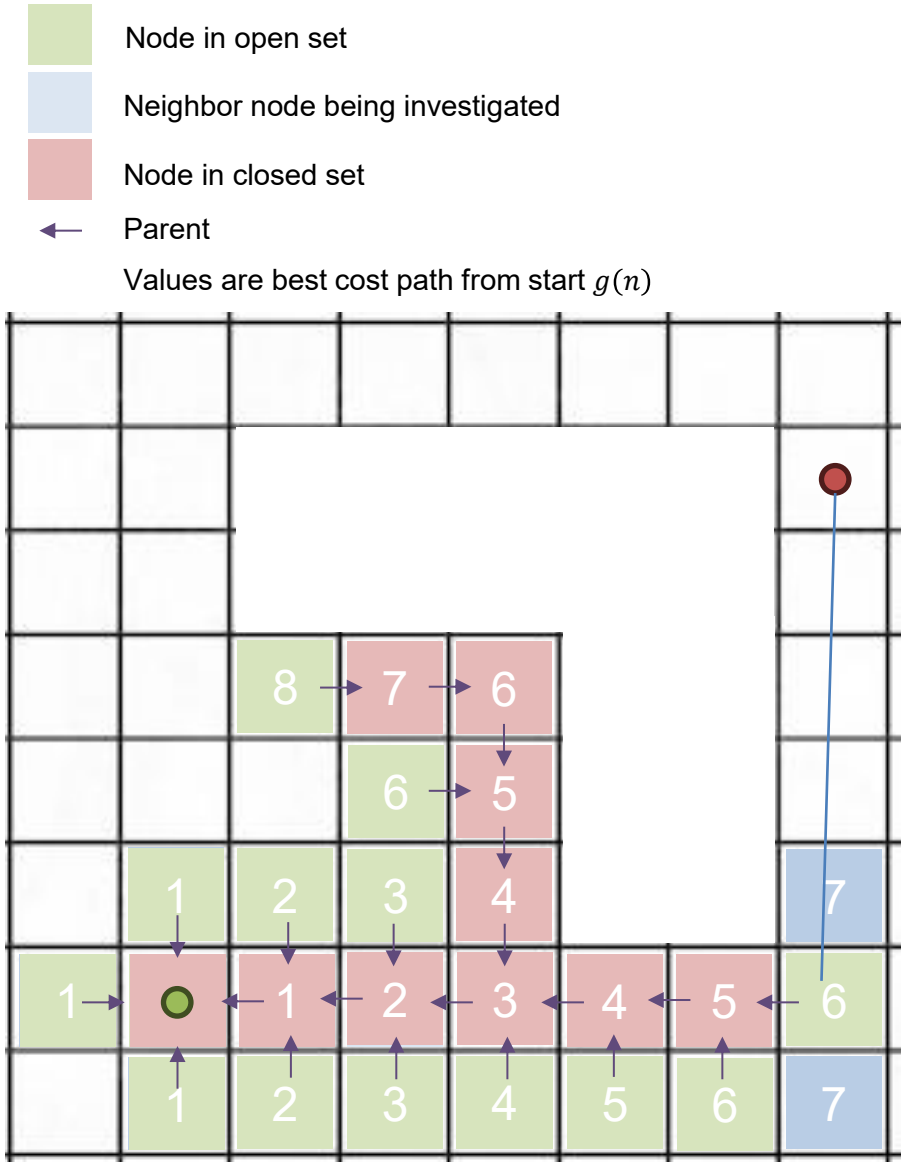
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



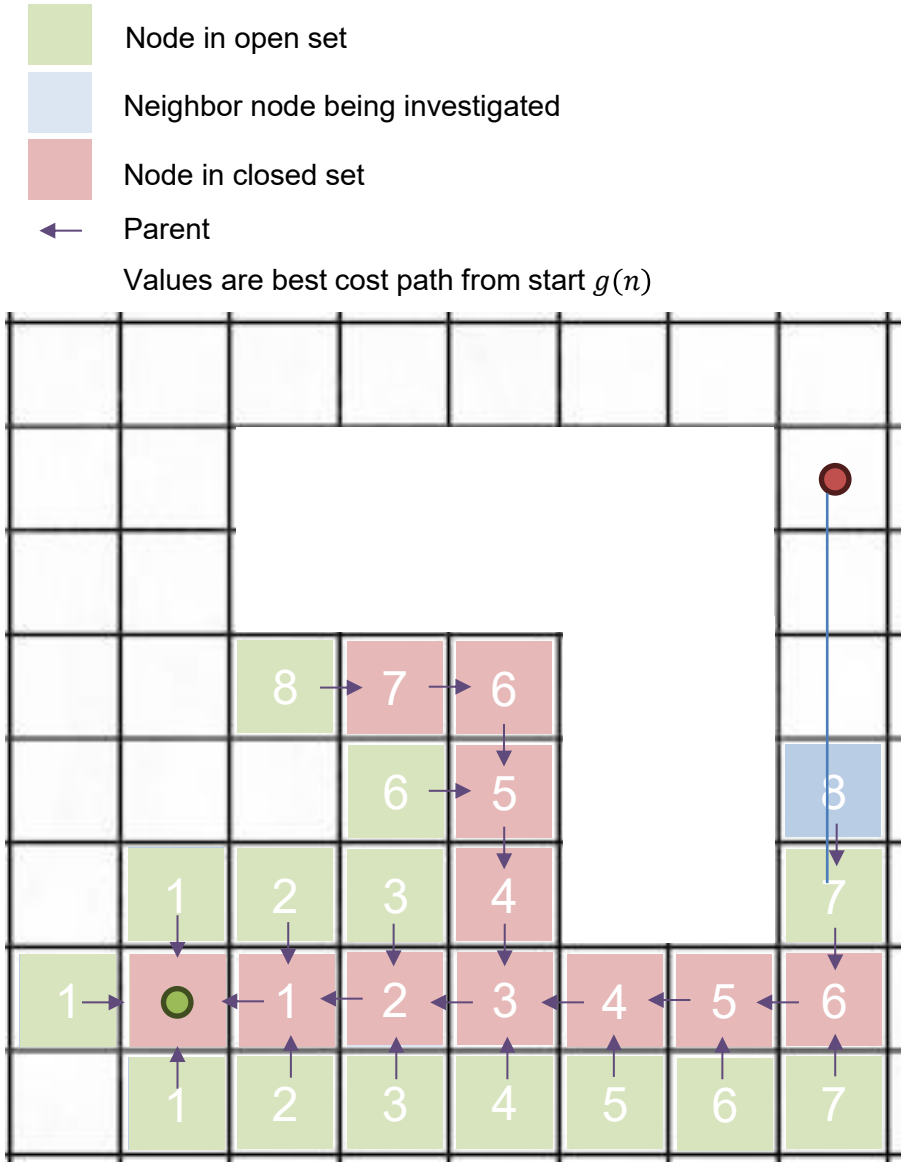
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



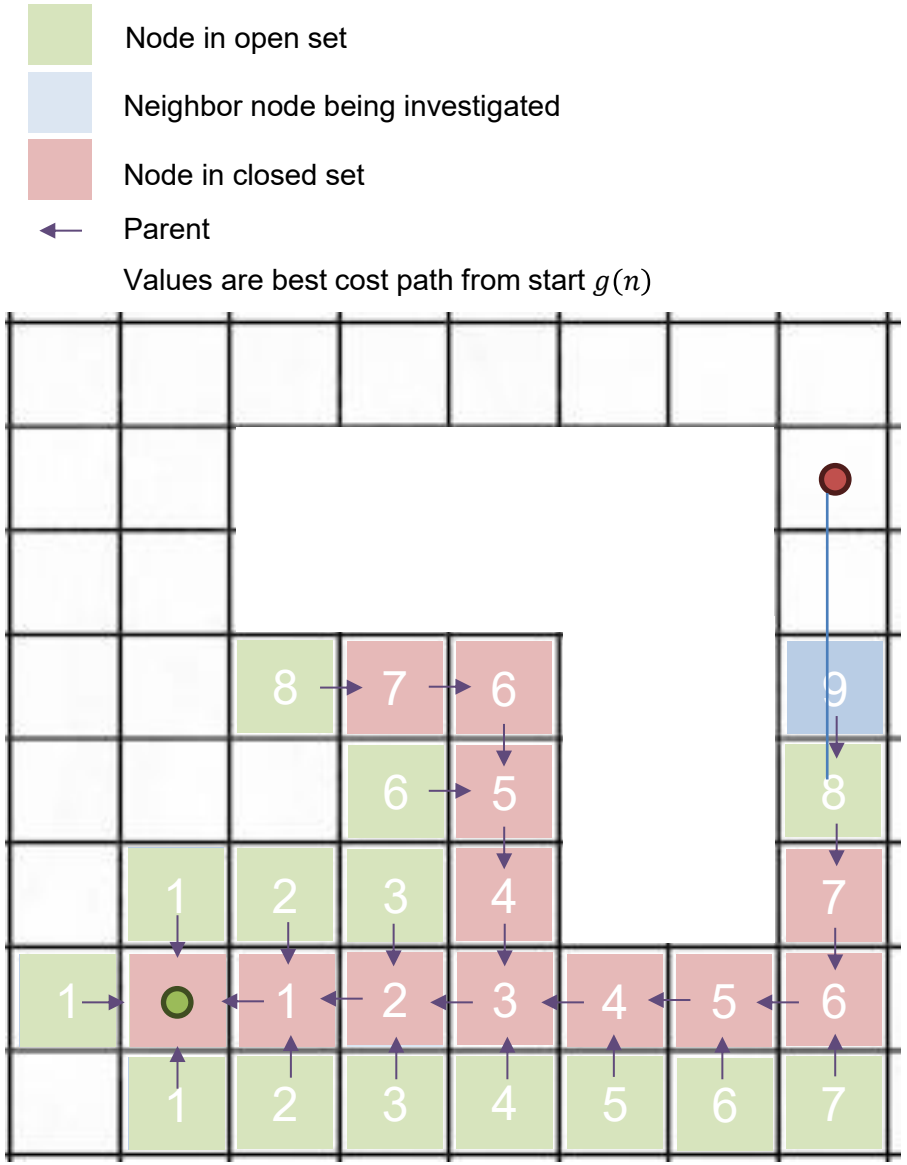
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



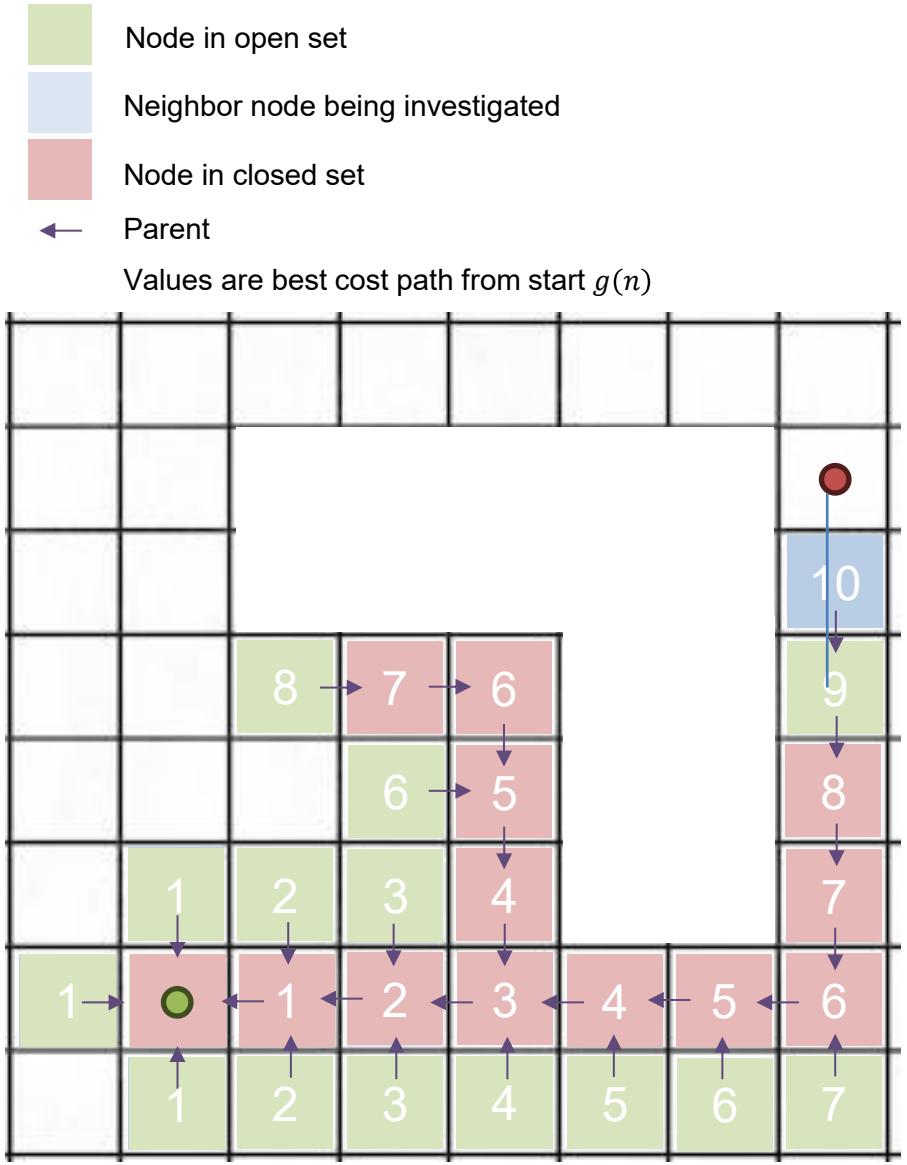
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



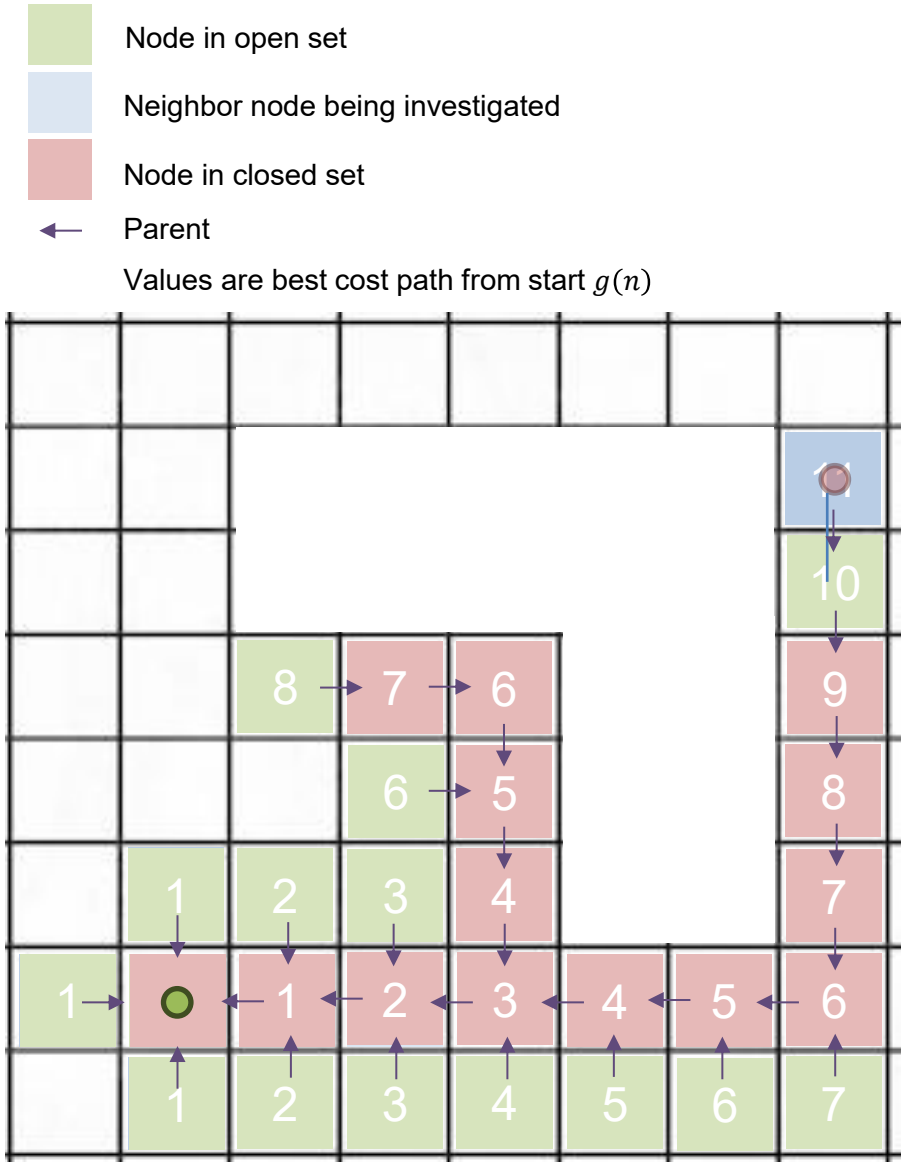
A* Example

- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



A* Example

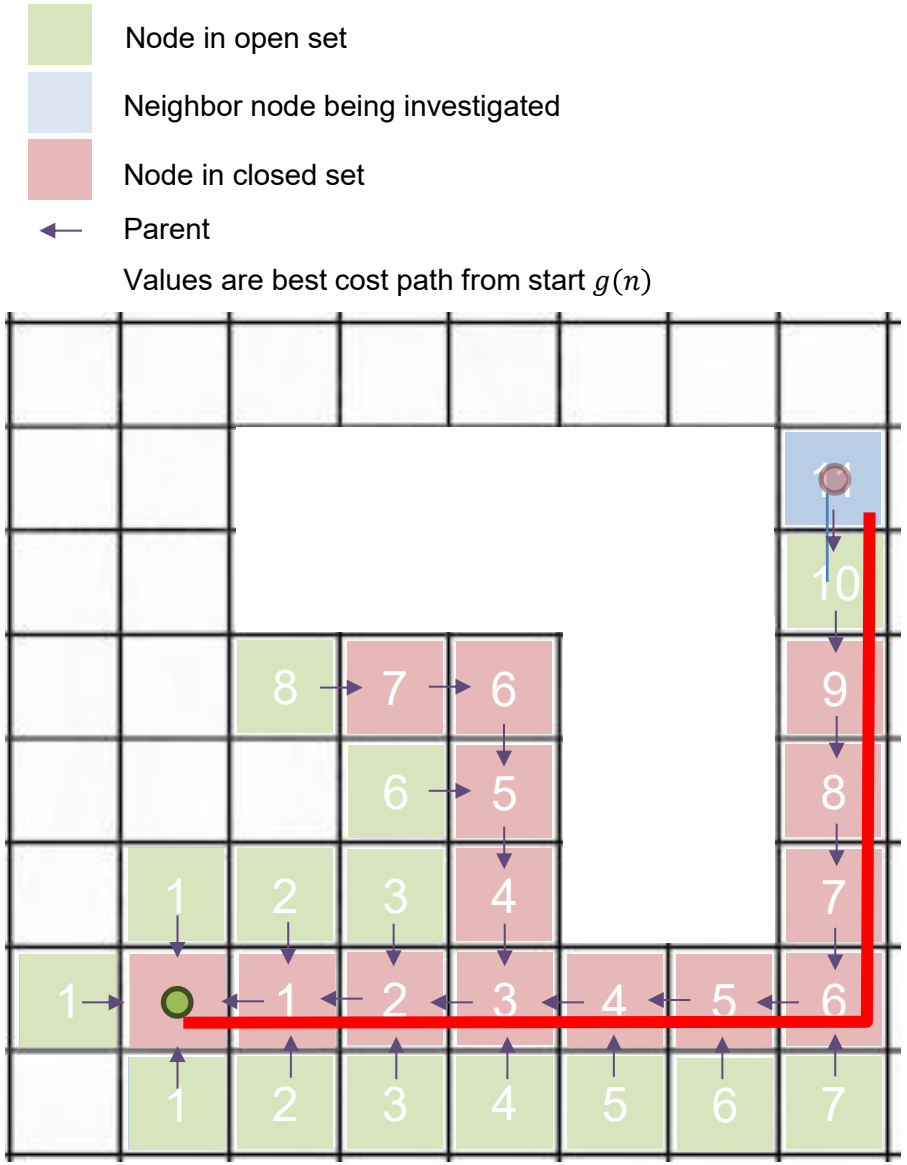
- For every node in the Open set, calculate $f(n)$
- Pick the lowest cost node n
- For all neighbors m of n not already in the Open or Closed sets
  - Add node m to Open set (might be more than one)
  - Update its cost from start ($g(m)$)
- For neighbors in the Open or Closed already
  - If there is shorter path via parent
    - Update shortest path $g(m)$
    - If node was on closed list, move it to Open
- Remove n from the open list to the close list since all neighbors have been explored
- Update parent of m to be n



A* Example

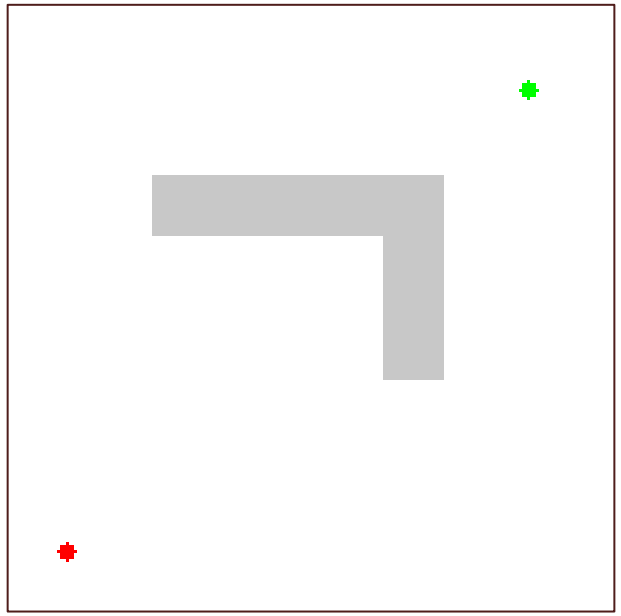
- Finally, if end location is one of the neighbours being investigated
  - End algorithm
  - Use chain of parent nodes to define path

- Algorithm only explored nodes which were perceived (via heuristic) to move towards goal.
- Not exhaustive search of every possible neighbour from current tree

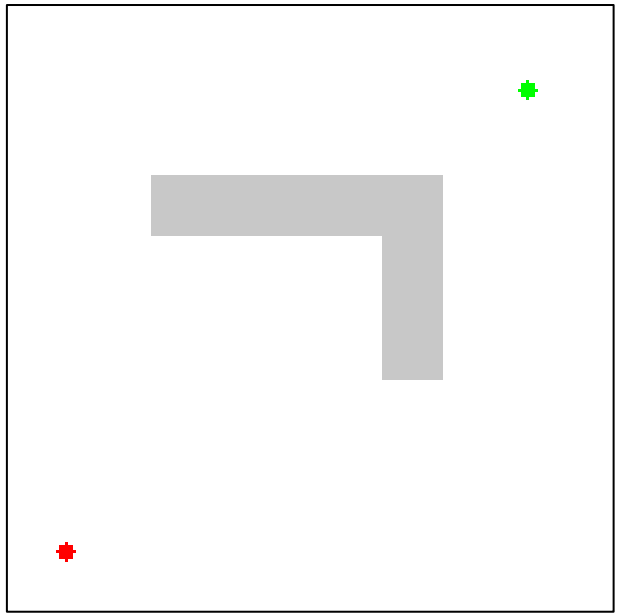


Dijkstra vs A*

Dijkstra - routes between nodes explored Breadth-first



A* - routes between nodes explored goal orientated

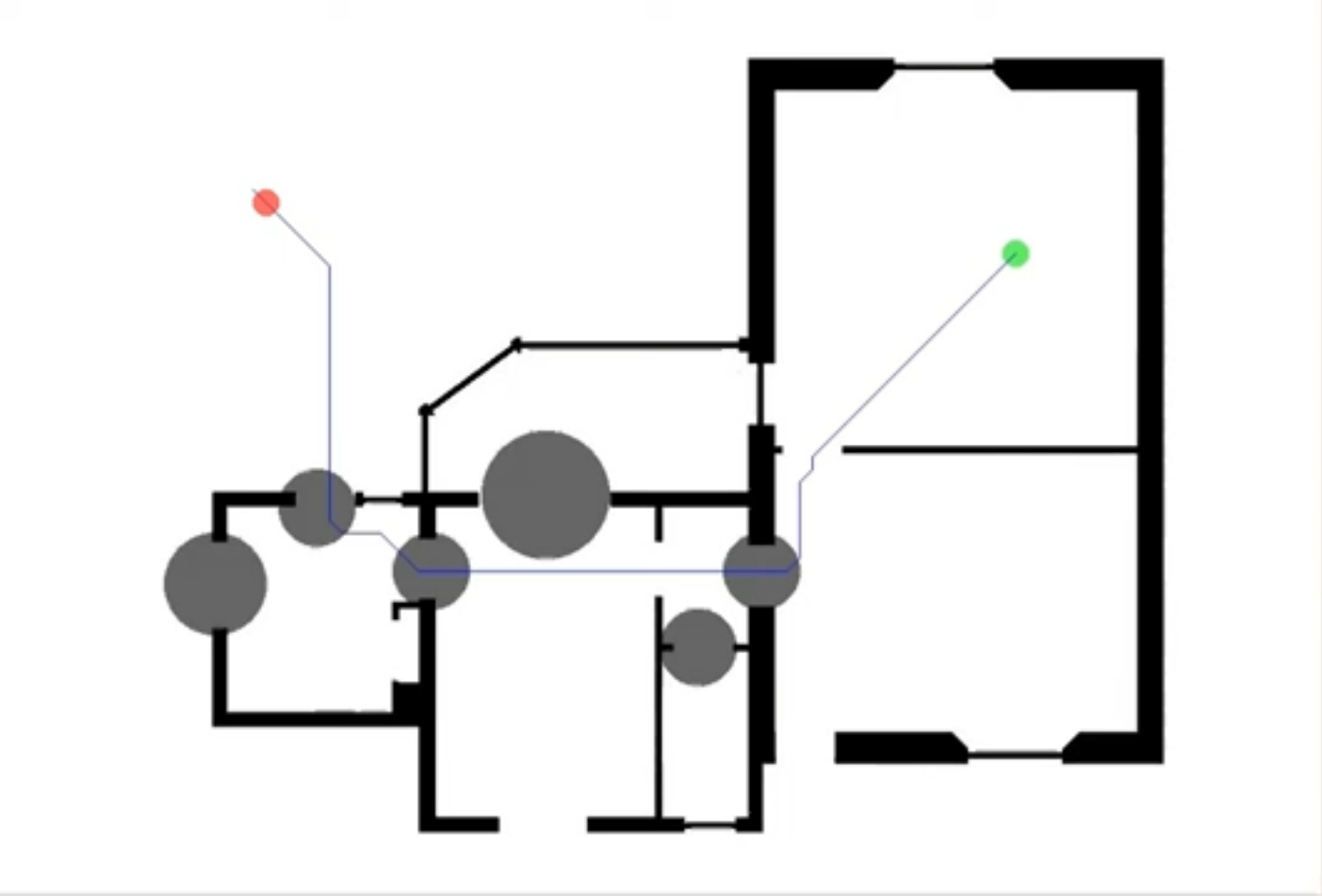


*Source is https://en.wikipedia.org/wiki/A*_search_algorithm*

What if we don't know the map?

- …or the environment is changing with time?
- The D* (d-star) algorithm
  - An extension of the A* method
  - Allows the costs associated with edges and heuristics to update over time
  - "Dynamic A*"
- Not just A* applied to updated map
  - D* allows for localized changes to small sections of a planned path
  - Reduces computational costs
- Useful for exploratory navigation algorithms when map is being created on the go (e.g. SLAM)

D* lite



S. Koenig and M. Likhachev. D* Lite. In Proceedings of the AAAI Conference of Artificial Intelligence (AAAI), 476-483, 2002.

Current research

- Optimised path panning for swarms
  - https://www.newscientist.com/article/2318637-watch-a-swarm-of-drones-navigate-a-forest-without-crashing/
  - Sharing map and information. Dynamic updates for non-static environment.
- High-speed (agile) path planning
  - https://rpg.ifi.uzh.ch/docs/Loquercio21_Science.pdf
  - Rapid updates. All-in-one approach to algorithm (sensor->path)

Intel RealSense Depth Camera D435

Coursework 2

- Coursework 1 marks and feedback released by end of this week
- Coursework 2 task and submission available on Canvas
  - Released at 2 pm today
  - Accompanying getting started lab class
    Monday 02-March 2-4pm
  - Computer Cluster 3 Engineering A
  - 15% of unit
- Similar marking scheme
  - Correctness, speed, efficiency of implementing A* original algorithms
  - Programming quality
  - Explanations